



KTH Royal Institute of Technology

Omogen Heap

Simon Lindholm, Johan Sannemo, Mårten Wiman

2026-08-28

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$
Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$
Law of tangents: $\frac{a+b}{a-b} = \frac{\tan \frac{\alpha+\beta}{2}}{\tan \frac{\alpha-\beta}{2}}$

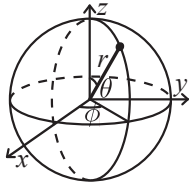
2.4.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180° , $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.4.3 Spherical coordinates



$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \arccos(z / \sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \operatorname{atan2}(y, x) \end{aligned}$$

2.5 Derivatives/Integrals

$$\begin{aligned} \frac{d}{dx} \arcsin x &= \frac{1}{\sqrt{1-x^2}} & \frac{d}{dx} \arccos x &= -\frac{1}{\sqrt{1-x^2}} \\ \frac{d}{dx} \tan x &= 1 + \tan^2 x & \frac{d}{dx} \arctan x &= \frac{1}{1+x^2} \\ \int \tan ax &= -\frac{\ln |\cos ax|}{a} & \int x \sin ax &= \frac{\sin ax - ax \cos ax}{a^2} \\ \int e^{-x^2} &= \frac{\sqrt{\pi}}{2} \operatorname{erf}(x) & \int x e^{ax} dx &= \frac{e^{ax}}{a^2} (ax - 1) \end{aligned}$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Sums

$$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

template .bashrc .vimrc hash troubleshoot

$$\begin{aligned} 1 + 2 + 3 + \dots + n &= \frac{n(n+1)}{2} \\ 1^2 + 2^2 + 3^2 + \dots + n^2 &= \frac{n(2n+1)(n+1)}{6} \\ 1^3 + 2^3 + 3^3 + \dots + n^3 &= \frac{n^2(n+1)^2}{4} \\ 1^4 + 2^4 + 3^4 + \dots + n^4 &= \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30} \end{aligned}$$

2.7 Series

$$\begin{aligned} e^x &= 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, (-\infty < x < \infty) \\ \ln(1+x) &= x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, (-1 < x \leq 1) \\ \sqrt{1+x} &= 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, (-1 \leq x \leq 1) \\ \sin x &= x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, (-\infty < x < \infty) \\ \cos x &= 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, (-\infty < x < \infty) \end{aligned}$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.8.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\operatorname{Bin}(n, p)$, $n = 1, 2, \dots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np, \sigma^2 = np(1-p)$$

$\operatorname{Bin}(n, p)$ is approximately $\operatorname{Po}(np)$ for small p .

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\operatorname{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1}, k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\operatorname{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$$

$$\mu = \lambda, \sigma^2 = \lambda$$

2.8.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\operatorname{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2}, \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\operatorname{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda}, \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi \mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i . π_j / π_i is the expected number of visits in state j between two visits in state i .

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i 's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k \rightarrow \infty} \mathbf{P}^k = \mathbf{1}\pi$.

A Markov chain is an A-chain if the states can be partitioned into two sets **A** and **G**, such that all states in **A** are absorbing ($p_{ii} = 1$), and all states in **G** leads to an absorbing state in **A**. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j , is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i , is $t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k$.

Data structures (3)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element and the number of elements smaller than a value. find_by_order is 0-indexed. Uses GNU policy-based data structures.
Time: $\mathcal{O}(\log N)$

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
```

```
template<class T, class Compare = less<T>>
using OrderStatisticTree = __gnu_pbds::tree<T, __gnu_pbds::
    null_type,
    Compare, __gnu_pbds::rb_tree_tag,
    __gnu_pbds::tree_order_statistics_node_update>;
```

```
void example() {
    OrderStatisticTree<int> t, t2; t.insert(8);
    auto it = t.insert(10).first;
    assert(it == t.lower_bound(9));
    assert(t.order_of_key(10) == 1);
    assert(t.order_of_key(11) == 2);
    assert(*t.find_by_order(0) == 8);
    t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

HashMap.h

Description: hash map with mostly the same API as unordered_map, but about 3x faster. Uses GNU policy-based data structures and a randomized hash.

```
78d7ba, 16 lines
struct CustomHash {
    static uint64_t split_mix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t RANDOM = chrono::steady_clock::now()
            .time_since_epoch().count();
        return split_mix64(x + RANDOM);
    }
};
#include <ext/pb_ds/assoc_container.hpp>
```

```
__gnu_pbds::gp_hash_table<ll, int, CustomHash> h;
```

SegmentTree.h

Description: 1-indexed max-tree with inclusive range bounds. can be changed by modifying value_type, f and UNIT.
Time: $\mathcal{O}(\log N)$

```
30d5a5, 20 lines
struct SegmentTree {
    typedef int value_type;
    static constexpr value_type UNIT = INT_MIN;
    value_type f(value_type a, value_type b) { return max(a, b);
    }
    vector<value_type> s; int n;
    SegmentTree(int n = 0, value_type def = UNIT) : s(2*n, def),
        n(n) {}
    void update(int pos, value_type val) {
        for (s[pos += n - 1] = val; pos /= 2;)
            s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
    }
    value_type query(int left, int right) { // query [left, right)
        value_type left_result = UNIT, right_result = UNIT;
        for (left += n - 1, right += n; left < right;
            left /= 2, right /= 2) {
            if (left % 2) left_result = f(left_result, s[left++]);
            if (right % 2) right_result = f(s[--right], right_result);
        }
        return f(left_result, right_result);
    }
};
```

LazySegmentTree.h

Description: segment tree with ability to add or set values of large intervals, and compute max of intervals. can be changed to other things. use with a bump allocator for better performance, and small_ptr or implicit indices to save memory.
Usage: LazySegmentTree tree(values); // public ranges are 1-indexed inclusive
Time: $\mathcal{O}(\log N)$.

```
"../various/BumpAllocator.h" 39bf17, 66 lines
const int INF = 1e9;
struct LazySegmentTree {
    LazySegmentTree *left_child = 0, *right_child = 0;
    int low, high, pending_set = INF, pending_add = 0, value = -
        INF;
    LazySegmentTree(int low, int high) : low(low), high(high) {}
    LazySegmentTree(const vector<int>& values) :
        LazySegmentTree(values, 1, (int)values.size() - 1) {}
```

```
LazySegmentTree(const vector<int>& values, int low, int high)
:
    low(low), high(high) {
    if (low < high) {
        int mid = low + (high - low) / 2;
        left_child = new LazySegmentTree(values, low, mid);
        right_child = new LazySegmentTree(values, mid + 1, high);
        value = max(left_child->value, right_child->value);
    }
    else value = values[low];
}

int query(int left, int right) {
    if (right < low || high < left) return -INF;
    if (left <= low && high <= right) return value;
    push();
    return max(left_child->query(left, right),
        right_child->query(left, right));
}

void set(int left, int right, int x) {
    if (right < low || high < left) return;
    if (left <= low && high <= right)
        pending_set = value = x, pending_add = 0;
    else {
        push();
        left_child->set(left, right, x);
        right_child->set(left, right, x);
        value = max(left_child->value, right_child->value);
    }
}

void add(int left, int right, int x) {
    if (right < low || high < left) return;
    if (left <= low && high <= right) {
        if (pending_set != INF) pending_set += x;
        else pending_add += x;
        value += x;
    }
    else {
        push();
        left_child->add(left, right, x);
        right_child->add(left, right, x);
        value = max(left_child->value, right_child->value);
    }
}

void push() {
    if (!left_child) {
        int mid = low + (high - low) / 2;
        left_child = new LazySegmentTree(low, mid);
        right_child = new LazySegmentTree(mid + 1, high);
    }
    if (pending_set != INF) {
        left_child->set(low, high, pending_set);
        right_child->set(low, high, pending_set);
        pending_set = INF;
    }
    else if (pending_add) {
        left_child->add(low, high, pending_add);
        right_child->add(low, high, pending_add);
        pending_add = 0;
    }
}
};
```

UnionFindRollback.h

Description: disjoint-set data structure with undo. elements are numbered 1..n. if undo is not needed, skip st, time() and rollback().
Usage: int t = uf.time(); ...; uf.rollback(t);
Time: $\mathcal{O}(\log(N))$

```
1eab25, 21 lines
struct RollbackUnionFind {
    vector<int> e; vector<pii> st;
```

```
RollbackUnionFind(int n) : e(n + 1, -1) {}
int size(int x) { return -e[find(x)]; }
int find(int x) { return e[x] < 0 ? x : find(e[x]); }
int time() { return (int)(st).size(); }
void rollback(int t) {
    for (int i = time(); i --> t;)
        e[st[i].first] = st[i].second;
    st.resize(t);
}
bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
}
};
```

SubMatrix.h

Description: calculate submatrix sums from a 1-indexed matrix using inclusive-inclusive corners. Row and column 0 are ignored.
Usage: SubMatrix<int> m(matrix);
m.sum(1, 1, 2, 2); // top left 4 elements
Time: $\mathcal{O}(N^2 + Q)$

```
template<class T>
struct SubMatrix {
    vector<vector<T>> p;
    SubMatrix(vector<vector<T>>& v) {
        int rows = (int)v.size() - 1, columns = (int)v[1].size() - 1;
        p.assign(rows + 1, vector<T>(columns + 1));
        for (int r = 1; r <= rows; ++r) for (int c = 1; c <= columns; ++c)
            p[r][c] = v[r][c] + p[r-1][c] + p[r][c-1] - p[r-1][c-1];
    }
    T sum(int upper, int left, int lower, int right) {
        return p[lower][right] - p[lower][left - 1]
            - p[upper - 1][right] + p[upper - 1][left - 1];
    }
};
```

Matrix.h

Description: basic operations on 1-indexed square matrices.
Usage: Matrix<int, 3> matrix;
matrix.d[1][1] = 1;
array<int, 4> vec = {0,1,2,3};
vec = (matrix^n) * vec;

```
template<class T, int SIZE> struct Matrix {
    array<array<T, SIZE + 1>, SIZE + 1> d{};
    Matrix operator*(const Matrix& m) const {
        Matrix a;
        for (int i = 1; i <= SIZE; ++i) for (int j = 1; j <= SIZE; ++j)
            for (int k = 1; k <= SIZE; ++k) a.d[i][k] += d[i][j] * m.d[j][k];
        return a;
    }
    array<T, SIZE + 1> operator*(const array<T, SIZE + 1>& vec) const {
        array<T, SIZE + 1> ret{};
        for (int i = 1; i <= SIZE; ++i) for (int j = 1; j <= SIZE; ++j)
            ret[i] += d[i][j] * vec[j];
        return ret;
    }
};
```

SubMatrix Matrix LineContainer Treap FenwickTree

```
Matrix operator^(ll p) const {
    assert(p >= 0);
    Matrix a, b(*this);
    for (int i = 1; i <= SIZE; ++i) a.d[i][i] = 1;
    while (p) {
        if (p&1) a = a*b;
        b = b*b;
        p >>= 1;
    }
    return a;
};
```

LineContainer.h

Description: container where you can add lines of the form $kx+m$, and query maximum values at points x . useful for dynamic programming (“convex hull trick”).
Time: $\mathcal{O}(\log N)$

```
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use INF = 1/.0, div(a,b) = a/b)
    static const ll INF = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = INF, 0;
        if (x->k == y->k) x->p = x->m > y->m ? INF : -INF;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```

Treap.h

Description: A short self-balancing tree. it acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.
Time: $\mathcal{O}(\log N)$

```
struct TreapNode {
    TreapNode *l = 0, *r = 0;
    int val, y, c = 1;
    TreapNode(int val) : val(val), y(rand()) {}
    void recalc();
};

int cnt(TreapNode* n) { return n ? n->c : 0; }
void TreapNode::recalc() { c = cnt(l) + cnt(r) + 1; }

template<class F> void each(TreapNode* n, F f) {
    if (n) { each(n->l, f); f(n->val); each(n->r, f); }
}
```

```
pair<TreapNode*, TreapNode*> split(TreapNode* n, int k) {
    if (!n) return {};
    if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
        auto [left, right] = split(n->l, k);
        n->l = right;
        n->recalc();
        return {left, n};
    } else {
        auto [left, right] = split(n->r, k - cnt(n->l) - 1); // and just "k"
        n->r = left;
        n->recalc();
        return {n, right};
    }
}
```

```
TreapNode* merge(TreapNode* l, TreapNode* r) {
    if (!l) return r;
    if (!r) return l;
    if (l->y > r->y) {
        l->r = merge(l->r, r);
        return l->recalc(), l;
    } else {
        r->l = merge(l, r->l);
        return r->recalc(), r;
    }
}
```

```
TreapNode* insert_at_internal(TreapNode* t, TreapNode* n, int pos) {
    auto [l,r] = split(t, pos);
    return merge(merge(l, n), r);
}
```

```
TreapNode* insert_at(TreapNode* tree, TreapNode* node, int position) {
    return insert_at_internal(tree, node, position - 1);
}
```

```
// move the 1-indexed inclusive range [left, right] to position target
void move(TreapNode*& t, int left, int right, int target) {
    int l = left - 1, r = right, k = target - 1;
    TreapNode *a, *b, *c;
    tie(a,b) = split(t, l); tie(b,c) = split(b, r - 1);
    if (k <= l) t = merge(insert_at_internal(a, b, k), c);
    else t = merge(a, insert_at_internal(c, b, k - r));
}
```

FenwickTree.h

Description: computes 1-indexed inclusive prefix sums and updates single elements, taking the difference between the old and new value.
Time: both operations are $\mathcal{O}(\log N)$.

```
struct FenwickTree {
    vector<ll> s;
    FenwickTree(int n) : s(n + 1) {}
    void update(int pos, ll dif) { // a[pos] += dif, pos is 1-indexed
        for (; pos < (int)s.size(); pos += pos & -pos) s[pos] += dif;
    }
    ll query(int pos) { // sum of values in [1, pos]
        ll res = 0;
        for (; pos > 0; pos -= pos & -pos) res += s[pos];
        return res;
    }
    int lower_bound(ll sum) { // min pos such that sum [1, pos] >= sum
}
```

```
// returns n + 1 if no prefix works, or 0 for the empty
// prefix.
if (sum <= 0) return 0;
int pos = 0;
int pw = 1;
while (pw < (int)s.size()) pw <= 1;
for (; pw; pw >= 1) {
    if (pos + pw < (int)s.size() && s[pos + pw] < sum)
        pos += pw, sum -= s[pos];
}
return pos + 1;
}
};
```

FenwickTree2d.h

Description: computes sums a[i,j] for all 1 <= i <= I, 1 <= j <= J, and increases single elements a[i,j]. requires that the elements to be updated are known in advance (call fake_update() before init()).
Time: $\mathcal{O}(\log^2 N)$. (use persistent segment trees for $\mathcal{O}(\log N)$.)

```
"FenwickTree.h" 2e940c, 22 lines

struct FenwickTree2D {
    vector<vector<int>>> ys; vector<FenwickTree> ft;
    FenwickTree2D(int max_x) : ys(max_x + 1) {}
    void fake_update(int x, int y) {
        for (; x < (int)ys.size(); x += x & -x) ys[x].push_back(y);
    }
    void init() {
        for (vector<int>& v : ys) sort(begin(v), end(v)), ft.emplace_back((int)(v).size());
    }
    int ind(int x, int y) {
        return (int)(lower_bound(begin(ys[x]), end(ys[x]), y) - ys[x].begin());
    }
    void update(int x, int y, ll dif) {
        for (; x < (int)ys.size(); x += x & -x) ft[x].update(ind(x, y) + 1, dif);
    }
    ll query(int x, int y) {
        ll sum = 0;
        for (; x; x -= x & -x) sum += ft[x].query(ind(x, y + 1));
        return sum;
    }
};
```

RMQ.h

Description: range minimum queries on an array. returns min(V[a], V[a+1], ... V[b]) in constant time. Input and query positions are 1-indexed and V[0] is ignored.
Usage: RangeMinimumQuery RangeMinimumQuery(values);
RangeMinimumQuery.query(inclusive, inclusive); // 1-indexed
Time: $\mathcal{O}(|V| \log |V| + Q)$

```
template<class T>
struct RangeMinimumQuery {

private:
    vector<vector<T>>> jmp;

public:
    RangeMinimumQuery(const vector<T>& values) :
        jmp(1, vector<T>(next(begin(values)), end(values))) {
        for (int pw = 1, k = 1; pw * 2 <= (int)jmp[0].size(); pw *= 2, ++k) {
            jmp.emplace_back((int)jmp[0].size() - pw * 2 + 1);
            for (int j = 0; j < ((int)(jmp[k]).size()); ++j)
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
};
```

```
T query(int left, int right) {
    assert(1 <= left && left <= right && right <= (int)jmp[0].size());
    --left;
    int depth = 31 - __builtin_clz(right - left);
    return min(jmp[depth][left], jmp[depth][right - (1 << depth)]);
}
};
```

MoQueries.h

Description: answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. if values are on tree edges, change step to add/remove the edge (a,c) and remove the initial add call (but keep in).
Time: $\mathcal{O}(N\sqrt{Q})$

```
void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vector<int> mo(vector<pii> queries) {
    int left = 1, right = 0, block_size = 350; // ~N/sqrt(Q)
    vector<int> order((int)queries.size()), result = order;
    auto key = [&](pii query) {
        int block = query.first / block_size;
        return pii(block, query.second ^ -(block & 1));
    };
    iota(begin(order), end(order), 0);
    sort(begin(order), end(order), [&](int a, int b) {
        return key(queries[a]) < key(queries[b]);
    });
    for (int qi : order) {
        pii q = queries[qi];
        while (left > q.first) add(--left, 0);
        while (right < q.second) add(++right, 1);
        while (left < q.first) del(left++, 0);
        while (right > q.second) del(right--, 1);
        result[qi] = calc();
    }
    return result;
}
```

```
vector<int> mo_tree(vector<array<int, 2>> queries, vector<vector<int>>& ed, int root=1){
    int n = (int)ed.size() - 1, timer = 0, pos[2] = {root, root}, block_size = 350;
    vector<int> order((int)queries.size()), result = order, index(n + 1), path(n + 1);
    vector<int> left(n + 1), right(n + 1), in(n + 1), par(n + 1);
    add(root, 0), in[root] = 1;
    auto dfs = [&](int x, int p, int dep, auto& f) -> void {
        par[x] = p;
        left[x] = timer;
        if (dep) index[x] = ++timer;
        for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
        if (!dep) index[x] = ++timer;
        right[x] = timer;
    };
    dfs(root, -1, 0, dfs);
    auto key = [&](array<int, 2> query) {
        int block = index[query[0]] / block_size;
        return pii(block, index[query[1]] ^ -(block & 1));
    };
    iota(begin(order), end(order), 0);
    sort(begin(order), end(order), [&](int a, int b) {
        return key(queries[a]) < key(queries[b]);
    });
    for (int qi : order) for (int end = 0; end < 2; ++end) {
```

```
int &a = pos[end], b = queries[qi][end], i = 0;
auto step = [&](int c) {
    if (in[c]) { del(a, end); in[a] = 0; }
    else { add(c, end); in[c] = 1; }
    a = c;
};
while (!(left[b] <= left[a] && right[a] <= right[b]))
    path[++i] = b, b = par[b];
while (a != b) step(par[a]);
while (i) step(path[i--]);
if (end) result[qi] = calc();
}
return result;
}
```

Numerical (4)

4.1 Polynomials and recurrences

Polynomial.h

```
struct Polynomial {
    vector<double> a;
    double operator()(double x) const {
        double val = 0;
        for (int i = (int)(a).size(); i--;) (val *= x) += a[i];
        return val;
    }
    void diff() {
        for (int i = 1; i < ((int)(a).size()); ++i) a[i-1] = i*a[i];
        a.pop_back();
    }
    void divroot(double x0) {
        double b = a.back(), c; a.back() = 0;
        for(int i=(int)(a).size()-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
        a.pop_back();
    }
};
```

PolyRoots.h

Description: finds the real roots to a polynomial.
Usage: poly_roots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0
Time: $\mathcal{O}(n^2 \log(1/\epsilon))$

```
"Polynomial.h" 565a20, 23 lines

vector<double> poly_roots(Polynomial p, double xmin, double xmax) {
    if ((int)(p.a).size() == 2) { return {-p.a[0]/p.a[1]}; }
    vector<double> ret;
    Polynomial der = p;
    der.diff();
    auto dr = poly_roots(der, xmin, xmax);
    dr.push_back(xmin-1);
    dr.push_back(xmax+1);
    sort(begin(dr), end(dr));
    for (int i = 0; i < ((int)(dr).size()-1); ++i) {
        double l = dr[i], h = dr[i+1];
        bool sign = p(l) > 0;
        if (sign ^ (p(h) > 0)) {
            for (int it = 0; it < (60); ++it) { // while (h - l > 1e-8)
                double m = (l + h) / 2, f = p(m);
                if ((f <= 0) ^ sign) l = m;
                else h = m;
            }
            ret.push_back((l + h) / 2);
        }
    }
```

```
    }
    return ret;
}
```

PolyInterpolate.h

Description: given n points $(x[i], y[i])$, computes an $n-1$ -degree polynomial p that passes through them: $p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}$. for numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$.
Time: $\mathcal{O}(n^2)$

```
vector<double> interpolate(vector<double> x, vector<double> y,
    int n) {
    vector<double> res(n), temp(n);
    for (int k = 0; k < (n-1); ++k) for (int i = k+1; i < (n); ++i)
        y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0; temp[0] = 1;
    for (int k = 0; k < (n); ++k) for (int i = 0; i < (n); ++i) {
        res[i] += y[k] * temp[i];
        swap(last, temp[i]);
        temp[i] -= last * x[k];
    }
    return res;
}
```

BerlekampMassey.h

Description: recovers any n -order linear recurrence relation from the first $2n$ terms of the recurrence. useful for guessing linear recurrences after brute-forcing the first terms. should work on any field, but numerical stability for floats is not guaranteed. output will have size $\leq n$.
Usage: berlekamp_massey({0, 1, 1, 3, 5, 11}, MOD) // {1, 2}
Time: $\mathcal{O}(N^2)$

```
vector<ll> berlekamp_massey(vector<ll> sequence, ll modulus) {
    int n = (int)sequence.size(), length = 0, shift = 0;
    vector<ll> current(n), previous(n), backup;
    current[0] = previous[0] = 1;
    auto mod_pow = [&](ll base, ll exponent) {
        ll result = 1;
        for (; exponent; exponent /= 2, base = base * base % modulus)
            if (exponent & 1) result = result * base % modulus;
        return result;
    };

    ll b = 1;
    for (int i = 0; i < n; ++i) { ++shift;
        ll d = sequence[i] % modulus;
        for (int j = 1; j < length + 1; ++j)
            d = (d + current[j] * sequence[i - j]) % modulus;
        if (!d) continue;
        backup = current;
        ll coefficient = d * mod_pow(b, modulus - 2) % modulus;
        for (int j = shift; j < n; ++j)
            current[j] = (current[j]
                - coefficient * previous[j - shift]) % modulus;
        if (2 * length > i) continue;
        length = i + 1 - length;
        previous = backup; b = d; shift = 0;
    }

    current.resize(length + 1); current.erase(current.begin());
    for (ll& x : current) x = (modulus - x) % modulus;
    return current;
}
```

LinearRecurrence.h

Description: generates the k 'th term of an n -order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. faster than matrix multiplication. useful together with berlekamp-massey.
Usage: linear_rec({0, 1}, {1, 1}, k) // k 'th fibonacci number
Time: $\mathcal{O}(n^2 \log k)$

```
ll linear_rec(vector<ll> sequence, vector<ll> transition, ll k)
{
    int n = (int)transition.size();

    auto combine = [&](vector<ll> a, vector<ll> b) {
        vector<ll> res(n * 2 + 1);
        for (int i = 0; i < (n+1); ++i) for (int j = 0; j < (n+1); ++j)
            res[i + j] = (res[i + j] + a[i] * b[j]) % MOD;
        for (int i = 2 * n; i > n; --i) for (int j = 0; j < (n); ++j)
            res[i - 1 - j] = (res[i - 1 - j] + res[i] * transition[j]) % MOD;
        res.resize(n + 1);
        return res;
    };

    vector<ll> pol(n + 1), e(pol);
    pol[0] = e[1] = 1;

    for (++k; k; k /= 2) {
        if (k % 2) pol = combine(pol, e);
        e = combine(e, e);
    }

    ll res = 0;
    for (int i = 0; i < (n); ++i)
        res = (res + pol[i + 1] * sequence[i]) % MOD;
    return res;
}
```

4.2 Optimization

GoldenSectionSearch.h

Description: finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. the maximum error in the result is eps . works equally well for maximization with a small change in the code. see ternary_search.h in the various chapter for a discrete version.
Usage: double func(double x) { return 4+x+.3*x*x; }
double xmin = gss(-1000, 1000, func);
Time: $\mathcal{O}(\log((b-a)/\epsilon))$

```
double gss(double a, double b, double (*f)(double)) {
    double r = (sqrt(5)-1)/2, eps = 1e-7;
    double x1 = b - r*(b-a), x2 = a + r*(b-a);
    double f1 = f(x1), f2 = f(x2);
    while (b-a > eps)
        if (f1 < f2) { //change to > to find maximum
            b = x2; x2 = x1; f2 = f1;
            x1 = b - r*(b-a); f1 = f(x1);
        } else {
            a = x1; x1 = x2; f1 = f2;
            x2 = a + r*(b-a); f2 = f(x2);
        }
    return a;
}
```

HillClimbing.h

Description: poor man's optimization for unimodal functions.

```
template<class F>
```

```
pair<double, array<double, 2>> hill_climb(array<double, 2>
    start, F f) {
    pair<double, array<double, 2>> cur(f(start), start);
    for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
        for (int j = 0; j < (100); ++j) for (int dx = -1; dx < (2); ++dx) for (int dy = -1; dy < (2); ++dy) {
            array<double, 2> p = cur.second;
            p[0] += dx*jmp;
            p[1] += dy*jmp;
            cur = min(cur, make_pair(f(p), p));
        }
    }
    return cur;
}
```

Integrate.h

Description: simple integration of a function over an interval using simpson's rule. the error should be proportional to h^4 , although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

```
template<class F>
double integrate(double a, double b, F f, const int n = 1000) {
    double h = (b - a) / 2 / n, v = f(a) + f(b);
    for (int i = 1; i < (n*2); ++i)
        v += f(a + i*h) * (i&1 ? 4 : 2);
    return v * h / 3;
}
```

IntegrateAdaptive.h

Description: fast integration using an adaptive simpson's rule.
Usage: double sphere.volume = integrate(-1, 1, [](double x) { return integrate(-1, 1, [&](double y) { return integrate(-1, 1, [&](double z) { return x*x + y*y + z*z < 1; });});});

```
typedef double d;
c2bd30, 21 lines
```

```
typedef double d;
```

```
template <class F>
d integrate_rec(F& f, d a, d b, d eps, d estimate) {
    auto simpson = [&](d left, d right) {
        return (f(left) + 4 * f((left + right) / 2) + f(right))
            * (right - left) / 6;
    };
    d c = (a + b) / 2;
    d left_estimate = simpson(a, c), right_estimate = simpson(c, b);
    d refined = left_estimate + right_estimate;
    if (abs(refined - estimate) <= 15 * eps || b - a < 1e-10)
        return refined + (refined - estimate) / 15;
    return integrate_rec(f, a, c, eps / 2, left_estimate)
        + integrate_rec(f, c, b, eps / 2, right_estimate);
}

template<class F>
d integrate(d a, d b, F f, d eps = 1e-8) {
    d estimate = (f(a) + 4 * f((a + b) / 2) + f(b)) * (b - a) / 6;
    return integrate_rec(f, a, b, eps, estimate);
}
```

Simplex.h

Description: solves a general linear maximization problem: maximize $c^T x$ subject to $ax \leq b, x \geq 0$. returns -INF if there is no solution, INF if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. matrix, b, c and the returned x are 1-indexed with index 0 unused. the output vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). numerical stability is not guaranteed. for better performance, define variables such that $x = 0$ is viable.

Usage: vector<vector<scalar>> matrix = {{}, {0,1,-1}, {0,-1,1}, {0,-1,-2}};
 vector<scalar> b = {0,1,-4}, c = {0,-1,-1}, x;
 scalar val = LinearProgramSolver(matrix, b, c).solve(x);
Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation.
 $\mathcal{O}(2^n)$ in the general case.

230f44, 79 lines

```
typedef double scalar; // long double, rational, double + modP
>...
```

```
const scalar EPS = 1e-8, INF = 1/.0;
```

```
struct LinearProgramSolver {
private:
  int m, n;
  vector<int> non_basic, basic;
  vector<vector<scalar>> tableau;

  void pivot(int r, int s) {
    scalar *a = tableau[r].data(), inv = 1 / a[s];
    for (int i = 0; i < (m+2); ++i) if (i != r && abs(tableau[i][s]) > EPS) {
      scalar *b = tableau[i].data(), inv2 = b[s] * inv;
      for (int j = 0; j < (n+2); ++j) b[j] -= a[j] * inv2;
      b[s] = a[s] * inv2;
    }
    for (int j = 0; j < (n+2); ++j) if (j != s) tableau[r][j] *= inv;
    for (int i = 0; i < (m+2); ++i) if (i != r) tableau[i][s] *= -inv;
    tableau[r][s] = inv;
    swap(basic[r], non_basic[s]);
  }
}
```

```
bool simplex(int phase) {
  int x = m + phase - 1;
  for (;;) {
    int s = -1;
    for (int j = 0; j < n + 1; ++j) if (non_basic[j] != -phase) {
      if (s == -1 || make_pair(tableau[x][j], non_basic[j]) < make_pair(tableau[x][s], non_basic[s])) s = j;
    }
    if (tableau[x][s] >= -EPS) return true;
    int r = -1;
    for (int i = 0; i < (m); ++i) {
      if (tableau[i][s] <= EPS) continue;
      if (r == -1 || make_pair(tableau[i][n+1] / tableau[i][s], basic[i]) < make_pair(tableau[r][n+1] / tableau[r][s], basic[r])) r = i;
    }
    if (r == -1) return false;
    pivot(r, s);
  }
}
```

```
public:
  LinearProgramSolver(const vector<vector<scalar>>& matrix,
    const vector<scalar>& b, const vector<scalar>& c) :
    m((int)b.size() - 1), n((int)c.size() - 1), non_basic(n+1),
    basic(m), tableau(m+2, vector<scalar>(n+2)) {
    for (int i = 0; i < m; ++i) for (int j = 0; j < n; ++j)
      tableau[i][j] = matrix[i+1][j+1];
    for (int i = 0; i < m; ++i) {
      basic[i] = n+i; tableau[i][n] = -1; tableau[i][n+1] = b[i+1];
    }
    for (int j = 0; j < n; ++j) {
```

```
    non_basic[j] = j; tableau[m][j] = -c[j+1];
  }
  non_basic[n] = -1; tableau[m+1][n] = 1;
}

scalar solve(vector<scalar>& x) {
  int r = 0;
  for (int i = 1; i < (m); ++i) if (tableau[i][n+1] < tableau[r][n+1]) r = i;
  if (tableau[r][n+1] < -EPS) {
    pivot(r, n);
    if (!simplex(2) || tableau[m+1][n+1] < -EPS) return -INF;
    for (int i = 0; i < (m); ++i) if (basic[i] == -1) {
      int s = 0;
      for (int j = 1; j < n + 1; ++j)
        if (make_pair(tableau[i][j], non_basic[j]) < make_pair(tableau[i][s], non_basic[s])) s = j;
      pivot(i, s);
    }
  }
  bool ok = simplex(1); x = vector<scalar>(n + 1);
  for (int i = 0; i < m; ++i) if (basic[i] < n)
    x[basic[i] + 1] = tableau[i][n+1];
  return ok ? tableau[m][n+1] : INF;
}
```

4.3 Matrices

Determinant.h

Description: calculates the determinant of a 1-indexed square matrix. Row and column 0 are ignored. Destroys the matrix.

Time: $\mathcal{O}(N^3)$

dd8b10, 15 lines

```
double det(vector<vector<double>>& a) {
  int n = (int)a.size() - 1; double res = 1;
  for (int i = 1; i <= n; ++i) {
    int b = i;
    for (int j = i+1; j <= n; ++j) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    for (int j = i+1; j <= n; ++j) {
      double v = a[j][i] / a[i][i];
      if (v != 0) for (int k = i+1; k <= n; ++k) a[j][k] -= v * a[i][k];
    }
  }
  return res;
}
```

IntDeterminant.h

Description: calculates the determinant of a 1-indexed square matrix using modular arithmetic. Row and column 0 are ignored. modulus can also be removed to get a pure-integer version.

Time: $\mathcal{O}(N^3)$

c0c873, 18 lines

```
const ll MOD = 12345;
ll det(vector<vector<ll>>& a) {
  int n = (int)a.size() - 1; ll ans = 1;
  for (int i = 1; i <= n; ++i) {
    for (int j = i+1; j <= n; ++j) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t) for (int k = i; k <= n; ++k)
          a[i][k] = (a[i][k] - a[j][k] * t) % MOD;
        swap(a[i], a[j]);
        ans *= -1;
      }
    }
  }
```

```
  }
  ans = ans * a[i][i] % MOD;
  if (!ans) return 0;
}
return (ans + MOD) % MOD;
}
```

SolveLinear.h

Description: solves $matrix * x = b$ for 1-indexed inputs. if there are multiple solutions, an arbitrary one is returned. returns rank, or -1 if no solutions. data in *matrix* and *b* is lost; x has index 0 unused.

Time: $\mathcal{O}(n^2m)$

cf77ee, 39 lines

```
const double EPS = 1e-12;
```

```
int solve_linear(vector<vector<double>>& matrix, vector<double>
  >& b, vector<double>& x) {
  int n = (int)matrix.size() - 1;
  int m = n ? (int)matrix[1].size() - 1 : 0;
  int rank = 0, br = 0, bc = 0;
  vector<int> col(m + 1); iota(next(begin(col)), end(col), 1);

  for (int i = 1; i <= min(n, m); ++i) {
    double v, bv = 0;
    for (int r = i; r <= n; ++r) for (int c = i; c <= m; ++c)
      if ((v = fabs(matrix[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= EPS) {
      for (int j = i; j <= n; ++j) if (fabs(b[j]) > EPS) return -1;
      break;
    }
    swap(matrix[i], matrix[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    for (int j = 1; j <= n; ++j) swap(matrix[j][i], matrix[j][bc]);
    bv = 1/matrix[i][i];
    for (int j = i+1; j <= n; ++j) {
      double fac = matrix[j][i] * bv;
      b[j] -= fac * b[i];
      for (int k = i+1; k <= m; ++k) matrix[j][k] -= fac*matrix[i][k];
    }
    rank++;
  }

  for (int i = rank + 1; i <= n; ++i) if (fabs(b[i]) > EPS)
    return -1;
  x.assign(m + 1, 0);
  for (int i = rank; i >= 1; --i) {
    b[i] /= matrix[i][i];
    x[col[i]] = b[i];
    for (int j = 1; j < i; ++j) b[j] -= matrix[j][i] * b[i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

SolveLinear2.h

Description: to get all uniquely determined values of *x* back from solve_linear, make the following changes:

"SolveLinear.h"

0d7689, 7 lines

```
for (int j = 0; j < (n); ++j) if (j != i) // instead of for (
  int j = i+1; j < (n); ++j)
// ... then at the end:
x.assign(m, undefined);
for (int i = 0; i < (rank); ++i) {
  for (int j = rank; j < (m); ++j) if (fabs(A[i][j]) > eps)
    goto fail;
```

```
    x[col[i]] = b[i] / A[i][i];
fail;; }
```

SolveLinearBinary.h

Description: solves $ax = b$ over $\mathbb{F}_2$ using 1-indexed rows and columns. if there are multiple solutions, one is returned arbitrarily. returns rank, or -1 if no solutions. destroys *matrix* and *b*.

Time: $\mathcal{O}(n^2m)$

295b5b, 33 lines

```
int solve_linear(vector<bitset<1000>>& matrix, vector<int>& b,
    bitset<1000> x, int m) {
    int n = (int)matrix.size() - 1, rank = 0, br;
    assert(m < (int)x.size());
    vector<int> col(m + 1); iota(next(begin(col)), end(col), 1);
    for (int i = 1; i <= min(n, m); ++i) {
        for (br=i; br<=n; ++br) if ((matrix[br] >> i).any()) break;
        if (br > n) {
            for (int j = i; j <= n; ++j) if(b[j]) return -1;
            break;
        }
        int bc = (int)matrix[br]._Find_next(i-1);
        swap(matrix[i], matrix[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        for (int j = 1; j <= n; ++j) if (matrix[j][i] != matrix[j][bc]) {
            matrix[j].flip(i); matrix[j].flip(bc);
        }
        for (int j = i+1; j <= n; ++j) if (matrix[j][i]) {
            b[j] ^= b[i];
            matrix[j] ^= matrix[i];
        }
        rank++;
    }

    x = bitset<1000>();
    for (int i = rank + 1; i <= n; ++i) if (b[i]) return -1;
    for (int i = rank; i >= 1; --i) {
        if (!b[i]) continue;
        x[col[i]] = 1;
        for (int j = 1; j < i; ++j) b[j] ^= matrix[j][i];
    }
    return rank; // (multiple solutions if rank < m)
}
```

MatrixInverse.h

Description: invert a 1-indexed matrix. returns rank; result is stored in *matrix* unless singular (rank < n). Row and column 0 are ignored. can easily be extended to prime moduli; for prime powers, repeatedly set $matrix^{-1} = matrix^{-1}(2I - AA^{-1}) \pmod{p^k}$ where $matrix^{-1}$ starts as the inverse of matrix mod p, and k is doubled in each step.

Time: $\mathcal{O}(n^3)$

c0284f, 36 lines

```
int mat_inv(vector<vector<double>>& matrix) {
    int n = (int)matrix.size() - 1; vector<int> col(n + 1);
    vector<vector<double>> tmp(n + 1, vector<double>(n + 1));
    for (int i = 1; i <= n; ++i) tmp[i][i] = 1, col[i] = i;

    for (int i = 1; i <= n; ++i) {
        int r = i, c = i;
        for (int j = i; j <= n; ++j) for (int k = i; k <= n; ++k)
            if (fabs(matrix[j][k]) > fabs(matrix[r][c]))
                r = j, c = k;
        if (fabs(matrix[r][c]) < 1e-12) return i - 1;
        matrix[i].swap(matrix[r]); tmp[i].swap(tmp[r]);
        for (int j = 1; j <= n; ++j)
            swap(matrix[j][i], matrix[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
    }
```

```
double v = matrix[i][i];
for (int j = i+1; j <= n; ++j) {
    double f = matrix[j][i] / v;
    matrix[j][i] = 0;
    for (int k = i+1; k <= n; ++k) matrix[j][k] -= f*matrix[i][k];
    for (int k = 1; k <= n; ++k) tmp[j][k] -= f*tmp[i][k];
}
for (int j = i+1; j <= n; ++j) matrix[i][j] /= v;
for (int j = 1; j <= n; ++j) tmp[i][j] /= v;
matrix[i][i] = 1;
}

for (int i = n; i > 1; --i) for (int j = 1; j < i; ++j) {
    double v = matrix[j][i];
    for (int k = 1; k <= n; ++k) tmp[j][k] -= v*tmp[i][k];
}

for (int i = 1; i <= n; ++i) for (int j = 1; j <= n; ++j)
    matrix[col[i]][col[j]] = tmp[i][j];
return n;
}
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves a 1-indexed equation system; all vectors have index 0 unused. Here $\text{diag}[i]$ is the diagonal, $\text{super}[i]$ is the coefficient at $(i, i + 1)$, and $\text{sub}[i]$ is the coefficient at $(i + 1, i)$. fails if the solution is not unique.

if $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither tr nor the check for $\text{diag}[i] == 0$ is needed.

Time: $\mathcal{O}(N)$

949498, 26 lines

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
    int n = (int)b.size() - 1; vector<int> tr(n + 1);
    for (int i = 1; i < n; ++i) {
        if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
            b[i+1] -= b[i] * diag[i+1] / super[i];
            if (i+2 <= n) b[i+2] -= b[i] * sub[i+1] / super[i];
            diag[i+1] = sub[i]; tr[++i] = 1;
        } else {
            diag[i+1] -= super[i]*sub[i]/diag[i];
            b[i+1] -= b[i]*sub[i]/diag[i];
        }
    }
    for (int i = n; i >= 1; --i) {
        if (tr[i]) {
            swap(b[i], b[i-1]);
            diag[i-1] = diag[i];
            b[i] /= super[i-1];
        } else {
            b[i] /= diag[i];
            if (i > 1) b[i-1] -= b[i]*super[i-1];
        }
    }
    return b;
}
```

4.4 Fourier transforms

FastFourierTransform.h

Description: $\text{fft}(a)$ computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. useful for convolution: $\text{conv}(a, b) = c$, where $c[x] = \sum a[i]b[x - i]$. for convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). otherwise, use NTT/fft_mod.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

9f330f, 35 lines

```
typedef complex<double> C;
void fft(vector<C>& a) {
    int n = (int)(a).size(), log_size = 31 - __builtin_clz(n);
    static vector<complex<long double>> roots(2, 1);
    static vector<C> rt(2, 1); // (^ 10% faster if double)
    for (static int k = 2; k < n; k *= 2) {
        roots.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        for (int i = k; i < (2*k); ++i)
            rt[i] = roots[i] = i&1 ? roots[i/2] * x : roots[i/2];
    }
    vector<int> rev(n);
    for (int i = 0; i < n; ++i) rev[i] = (rev[i / 2] | (i & 1) <<
        log_size) / 2;
    for (int i = 0; i < (n); ++i) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) for (int j = 0; j < (k); ++j) {
            C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
            a[i + j + k] = a[i + j] - z;
            a[i + j] += z;
        }
}

vector<double> conv(const vector<double>& a, const vector<double>& b) {
    if (a.empty() || b.empty()) return {};
    vector<double> res((int)(a).size() + (int)(b).size() - 1);
    int log_size = 32 - __builtin_clz((int)res.size()), n = 1 << log_size;
    vector<C> in(n), out(n);
    copy(begin(a), end(a), begin(in));
    for (int i = 0; i < ((int)(b).size()); ++i) in[i].imag(b[i]);
    fft(in);
    for (C& x : in) x *= x;
    for (int i = 0; i < (n); ++i) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    for (int i = 0; i < ((int)(res).size()); ++i) res[i] = imag(out[i]) / (4 * n);
    return res;
}
```

FastFourierTransformMod.h

Description: higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h"

7c2aa2, 22 lines

```
template<int MODULUS> vector<ll> conv_mod(const vector<ll>& a,
    const vector<ll>& b) {
    if (a.empty() || b.empty()) return {};
    vector<ll> res((int)(a).size() + (int)(b).size() - 1);
    int log_size=32-__builtin_clz((int)(res).size()), n=1<<log_size;
    int cut=int(sqrt(MODULUS));
    vector<C> left(n), right(n), outs(n), outl(n);
    for (int i = 0; i < ((int)(a).size()); ++i) left[i] = C((int)a[i] / cut, (int)a[i] % cut);
    for (int i = 0; i < ((int)(b).size()); ++i) right[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(left), fft(right);
    for (int i = 0; i < (n); ++i) {
        int j = -i & (n - 1);
        outl[j] = (left[i] + conj(left[j])) * right[i] / (2.0 * n);
        outs[j] = (left[i] - conj(left[j])) * right[i] / (2.0 * n) / 1i;
    }
```

```

}
fft(outl), fft(outs);
for (int i = 0; i < ((int) (res).size()); ++i) {
    ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
    ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
    res[i] = ((av % MODULUS * cut + bv) % MODULUS * cut + cv) %
        MODULUS;
}
return res;
}
```

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k , where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. useful for convolution modulo specific nice primes of the form 2^ab+1 , where the convolution result has size at most 2^a . for arbitrary modulo, see fft_mod. conv(a, b) = c, where $c[x] = \sum a[i]b[x-i]$. for manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. inputs must be in [0, mod).
Time: $\mathcal{O}(N \log N)$

```

2a42c7, 40 lines
const ll MOD = (119 << 23) + 1, ROOT = 62; // = 998244353
// for p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). the last two are > 10^9.
ll ntt_mod_pow(ll base, ll exponent) {
    ll result = 1;
    for (; exponent; exponent /= 2, base = base * base % MOD)
        if (exponent & 1) result = result * base % MOD;
    return result;
}
void ntt(vector<ll> &a) {
    int n = (int) (a).size(), log_size = 31 - __builtin_clz(n);
    static vector<ll> rt(2, 1);
    for (static int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        ll z[] = {1, ntt_mod_pow(ROOT, MOD >> s)};
        for (int i = k; i < (2*k); ++i) rt[i] = rt[i / 2] * z[i &
            1] % MOD;
    }
    vector<int> rev(n);
    for (int i = 0; i < n; ++i) rev[i] = (rev[i / 2] | (i & 1) <<
        log_size) / 2;
    for (int i = 0; i < (n); ++i) if (i < rev[i]) swap(a[i], a[
        rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) for (int j = 0; j < (k);
            ++j) {
            ll z = rt[j + k] * a[i + j + k] % MOD, &ai = a[i + j];
            a[i + j + k] = ai - z + (z > ai ? MOD : 0);
            ai += (ai + z >= MOD ? z - MOD : z);
        }
}
vector<ll> conv(const vector<ll> &a, const vector<ll> &b) {
    if (a.empty() || b.empty()) return {};
    int s = (int) (a).size() + (int) (b).size() - 1;
    int log_size = 32 - __builtin_clz(s), n = 1 << log_size;
    int inv = ntt_mod_pow(n, MOD - 2);
    vector<ll> left(a), right(b), out(n);
    left.resize(n), right.resize(n);
    ntt(left), ntt(right);
    for (int i = 0; i < (n); ++i)
        out[-i & (n - 1)] = (ll)left[i] * right[i] % MOD * inv %
            MOD;
    ntt(out);
    return {out.begin(), out.begin() + s};
}
```

FastSubsetTransform.h

Description: transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x\oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. the size of a must be a power of two.
Time: $\mathcal{O}(N \log N)$

```

d57f5f, 16 lines
void fast_subset_transform(vector<int>& a, bool inv) {
    for (int n = (int) (a).size(), step = 1; step < n; step *= 2)
        {
            for (int i = 0; i < n; i += 2 * step) for (int j = i; j < (
                i+step); ++j) {
                int &u = a[j], &v = a[j + step]; tie(u, v) =
                    inv ? pii(v - u, u) : pii(v, u + v); // AND
                    inv ? pii(v, u - v) : pii(u + v, u); // OR
                    pii(u + v, u - v); // XOR
            }
            if (inv) for (int& x : a) x /= (int) (a).size(); // XOR only
        }
    vector<int> conv(vector<int> a, vector<int> b) {
        fast_subset_transform(a, 0); fast_subset_transform(b, 0);
        for (int i = 0; i < ((int) (a).size()); ++i) a[i] *= b[i];
        fast_subset_transform(a, 1); return a;
    }
}
```

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

Description: operators for modular arithmetic. you need to set ModInt to some number first and then you can use the structure.

```

c51819, 18 lines
"euclid.h"
const ll MOD = 17; // change to something else
struct ModInt {
    ll x;
    ModInt(ll xx) : x(xx) {}
    ModInt operator+(ModInt b) { return ModInt((x + b.x) % MOD);
    }
    ModInt operator-(ModInt b) { return ModInt((x - b.x + MOD) %
        MOD); }
    ModInt operator*(ModInt b) { return ModInt((x * b.x) % MOD);
    }
    ModInt operator/(ModInt b) { return *this * invert(b); }
    ModInt invert(ModInt a) {
        ll x, y, g = euclid(a.x, MOD, x, y);
        assert(g == 1); return ModInt((x + MOD) % MOD);
    }
    ModInt operator^(ll e) {
        if (!e) return ModInt(1);
        ModInt r = *this ^ (e / 2); r = r * r;
        return e&1 ? *this * r : r;
    }
};
```

ModInverse.h

Description: pre-computation of modular inverses. assumes LIM ≤ mod and that mod is a prime.

```

a1a002, 3 lines
const ll mod = 1000000007, LIM = 200000;
ll* inv = new ll[LIM] - 1; inv[1] = 1;
for (int i = 2; i < (LIM); ++i) inv[i] = mod - (mod / i) * inv[
    mod % i] % mod;
```

ModPow.h

```

ebc734, 8 lines
const ll MOD = 1000000007;
```

```

ll mod_pow(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % MOD, e /= 2)
        if (e & 1) ans = ans * b % MOD;
    return ans;
}
```

ModLog.h

Description: returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. mod_log(a,l,m) can be used to calculate the order of a .

```

e2f351, 11 lines
Time:  $\mathcal{O}(\sqrt{m})$ 
ll mod_log(ll a, ll b, ll m) {
    ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> table;
    while (j <= n && (e = f = e * a % m) != b % m)
        table[e * b % m] = j++;
    if (e == b % m) return j;
    if (__gcd(m, e) == __gcd(m, b))
        for (int i = 2; i < (n+2); ++i) if (table.count(e = e * f %
            m))
            return n * i - table[e];
    return -1;
}
```

ModSum.h

Description: sums of mod'ed arithmetic progressions. $\text{modsum}(to, c, k, m) = \sum_{i=0}^{to-1} (ki+c)\%m$. divsum is similar but for floored division.
Time: $\log(m)$, with a large constant.

```

5c5bc5, 16 lines
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
```

```

ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m; c %= m;
    if (!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}
```

```

ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

```

06cbef, 11 lines
typedef unsigned long long ull;
ull mod_mul(ull a, ull b, ull modulus) {
    ll ret = a * b - modulus * ull(1.L / modulus * a * b);
    return ret + modulus * (ret < 0) - modulus * (ret >= (ll)
        modulus);
}
ull mod_pow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = mod_mul(b, b, mod), e /= 2)
        if (e & 1) ans = mod_mul(ans, b, mod);
    return ans;
}
```

ModSqrt.h

Description: tonelli-shanks algorithm for modular square roots. finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).

Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p	
"ModPow.h"	96f32c, 24 lines
<pre>ll sqrt(ll a, ll p) { a %= p; if (a < 0) a += p; if (a == 0) return 0; assert(mod_pow(a, (p-1)/2, p) == 1); // else no solution if (p % 4 == 3) return mod_pow(a, (p+1)/4, p); // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5 ll s = p - 1, n = 2; int r = 0, m; while (s % 2 == 0) ++r, s /= 2; while (mod_pow(n, (p - 1) / 2, p) != p - 1) ++n; ll x = mod_pow(a, (s + 1) / 2, p); ll b = mod_pow(a, s, p), g = mod_pow(n, s, p); for (; r = m) { ll t = b; for (m = 0; m < r && t != 1; ++m) t = t * t % p; if (m == 0) return x; ll gs = mod_pow(g, 1LL << (r - m - 1), p); g = gs * gs % p; x = x * gs % p; b = b * g % p; } }</pre>	

5.2 Primality

FastEratosthenes.h
Description: prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 ≈ 1.5s

	0aa312f, 20 lines
<pre>const int LIM = 1e6; bitset<LIM> is_prime; vector<int> eratosthenes() { const int block_size = (int)round(sqrt(LIM)), half_limit = LIM / 2; vector<int> pr = {2}, sieve(block_size+1); pr.reserve(int(LIM /log(LIM)*1.1)); vector<pii> cp; for (int i = 3; i <= block_size; i += 2) if (!sieve[i]) { cp.push_back({i, i * i / 2}); for (int j = i * i; j <= block_size; j += 2 * i) sieve[j] = 1; } for (int block_left = 1; block_left <= half_limit; block_left += block_size) { array<bool, block_size> block{}; for (auto &[p, idx] : cp) for (int i=idx; i < block_size+block_left; idx = (i+=p)) block[i-block_left] = 1; for (int i = 0; i < min(block_size, half_limit - block_left); ++i) if (!block[i]) pr.push_back((block_left + i) * 2 + 1); } for (int i : pr) is_prime[i] = 1; return pr; }</pre>	

MillerRabin.h	
Description: deterministic miller-rabin primality test. guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use python and extend A randomly. Time: 7 times the complexity of $a^b \bmod c$.	
"ModMulLL.h"	d798c5, 12 lines
<pre>bool is_prime(ull n) { if (n < 2 n % 6 % 4 != 1) return (n 1) == 3;</pre>	

<pre>ull bases[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022}, s = __builtin_ctzll(n-1), d = n >> s; for (ull a : bases) { // ^ count trailing zeroes ull p = mod_pow(a%n, d, n), i = s; while (p != 1 && p != n - 1 && a % n && i--) p = mod_mul(p, p, n); if (p != n-1 && i != s) return 0; } return 1;</pre>	

Factor.h
Description: pollard-rho randomized factorization algorithm. returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h"	ea7867, 18 lines
<pre>ull pollard(ull n) { ull x = 0, y = 0, t = 30, prd = 2, i = 1, q; auto f = [&](ull x) { return mod_mul(x, x, n) + i; }; while (t++ % 40 __gcd(prd, n) == 1) { if (x == y) x = ++i, y = f(x); if ((q = mod_mul(prd, max(x,y) - min(x,y), n))) prd = q; x = f(x), y = f(f(y)); } return __gcd(prd, n); } vector<ull> factor(ull n) { if (n == 1) return {}; if (is_prime(n)) return {n}; ull x = pollard(n); auto l = factor(x), r = factor(n / x); l.insert(l.end(), begin(r), end(r)); return l; }</pre>	

5.3 Divisibility

euclid.h
Description: finds two integers x and y , such that $ax + by = \gcd(a, b)$. if you just need gcd, use the built in _gcd instead. if a and b are coprime, then x is the inverse of $a \pmod b$.

	33ba8f, 5 lines
<pre>ll euclid(ll a, ll b, ll &x, ll &y) { if (!b) return x = 1, y = 0, a; ll d = euclid(b, a % b, y, x); return y -= a/b * x, d; }</pre>	

CRT.h
Description: chinese remainder theorem.
crt(a , m , b , n) computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. if $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. assumes $mn < 2^{62}$.
Time: $\log(n)$

"euclid.h"	04d93a, 7 lines
<pre>ll crt(ll a, ll m, ll b, ll n) { if (n > m) swap(a, b), swap(m, n); ll x, y, g = euclid(m, n, x, y); assert((a - b) % g == 0); // else no solution x = (b - a) % n * x % n / g * m + a; return x < 0 ? x + m*n/g : x; }</pre>	

5.3.1 Bézout’s identity

For $a \neq, b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h
Description: *euler’s ϕ* function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1, p$ prime $\rightarrow \phi(p^k) = (p - 1)p^{k-1}, m, n$ coprime $\rightarrow \phi(mn) = \phi(m)\phi(n)$. if $n = p_1^{k_1} p_2^{k_2} \dots p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} \dots (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$. $\sum_{d|n} \phi(d) = n, \sum_{1 \leq k \leq n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
euler’s thm: a, n coprime $\rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
fermat’s little thm: p prime $\rightarrow a^{p-1} \equiv 1 \pmod p \forall a$.

	51560d, 8 lines
<pre>const int LIM = 5000000; int phi[LIM]; void calculate_phi() { for (int i = 0; i < (LIM); ++i) phi[i] = i&1 ? i : i/2; for (int i = 3; i < LIM; i += 2) if (phi[i] == i) for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i; }</pre>	

5.4 Fractions

ContinuedFractions.h
Description: given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. it will obey $|p/q - x| \leq 1/q_n$. for consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. $(p_k/q_k$ alternates between $> x$ and $< x$.) if x is rational, y eventually becomes ∞ ; if x is the root of a degree 2 polynomial the a ’s eventually become cyclic.
Time: $\mathcal{O}(\log N)$

	768d27, 22 lines
<pre>typedef double d; // for N ~ 1e7; long double for N ~ 1e9 pair<ll, ll> approximate(d x, ll max_denominator) { ll last_p = 0, last_q = 1, p = 1, q = 0, inf = LLONG_MAX; d y = x; for (;) { ll lim = min(p ? (max_denominator-last_p) / p : inf, q ? (max_denominator-last_q) / q : inf), a = (ll)floor(y), b = min(a, lim), next_p = b*p + last_p, next_q = b*q + last_q; if (a > b) { // if b > a/2, we have a semi-convergent that gives us a // better approximation; if b = a/2, we *may* have one. // return {p, q} here for a more canonical approximation. return (abs(x - (d)next_p / (d)next_q) < abs(x - (d)p / (d)q)) ? make_pair(next_p, next_q) : make_pair(p, q); } if (abs(y = 1/(y - (d)a)) > 3*max_denominator) { return {next_p, next_q}; } last_p = p; p = next_p; last_q = q; q = next_q; } }</pre>	

FracBinarySearch.h

Description: given f and N , finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N$. you may want to throw an exception from f if it finds an exact solution, in which case N can be removed.
Usage: `frac_bs({}(Fraction f) { return f.p>=3*f.q; }, 10); //`
`{1,3}`
Time: $\mathcal{O}(\log(N))$

eb264d, 25 lines

```
struct Fraction { ll p, q; };

template<class F>
Fraction frac_bs(F f, ll max_denominator) {
    bool dir = 1, active = 1, previous_active = 1;
    Fraction lo{0, 1}, hi{1, 1}; // set hi to 1/0 to search (0,
        max_denominator]
    if (f(lo)) return lo;
    assert(f(hi));
    while (active || previous_active) {
        ll adv = 0, step = 1; // move hi if dir, else lo
        for (int si = 0; step; (step *= 2) >= si) {
            adv += step;
            Fraction mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
            if (abs(mid.p) > max_denominator || mid.q >
                max_denominator || dir == !f(mid)) {
                adv -= step; si = 2;
            }
        }
        hi.p += lo.p * adv;
        hi.q += lo.q * adv;
        dir = !dir;
        swap(lo, hi);
        active = previous_active; previous_active = !adv;
    }
    return dir ? hi : lo;
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$

with $m > n > 0, k > 0, m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

$\sum_{d|n} d = O(n \log \log n).$

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$

$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Description: permutation -> integer conversion. (not order preserving.)
integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$

55afb0, 6 lines

```
int perm_to_int(vector<int>& v) {
    int use = 0, i = 0, r = 0;
    for(int x:v) r = r * ++i + __builtin_popcount(use & ~(1<<x)),
        use |= 1 << x; // (note: minus, not ~!)
    return r;
}
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^\infty g_S(n) \frac{x^n}{n!} = \exp \left(\sum_{n \in S} \frac{x^n}{n} \right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.1.4 Burnside’s lemma

Given a group G of symmetries and a set X , the number of elements of X up to symmetry equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n, k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k - 1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	$\sim 2e5$	$\sim 2e8$

6.2.2 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \dots + n_1 p + n_0$ and $m = m_k p^k + \dots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod{p}$.

6.2.3 Binomials

multinomial.h

Description: computes $\binom{k_1 + \dots + k_n}{k_1, k_2, \dots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \dots k_n!}$.

b02f8c, 5 lines

```
ll multinomial(vector<int>& v) {
    ll c = 1, m = v.empty() ? 1 : v[0];
    for (int i = 1; i < ((int)(v).size()); ++i) for (int j = 0; j
        < (v[i]); ++j) c = c * ++m / (j+1);
    return c;
}
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).
 $B[0, \dots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \dots]$

Sums of powers:

$$\sum_{i=1}^n n^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\begin{aligned} \sum_{i=m}^\infty f(i) &= \int_m^\infty f(x) dx - \sum_{k=1}^\infty \frac{B_k}{k!} f^{(k-1)}(m) \\ &\approx \int_m^\infty f(x) dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m)) \end{aligned}$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k)=c(n-1,k-1)+(n-1)c(n-1,k),\; c(0,0)=1$$
$$\sum_{k=0}^nc(n,k)x^k=x(x+1)\dots(x+n-1)$$

$$c(8,k)=8,0,5040,13068,13132,6769,1960,322,28,1$$
$$c(n,2)=0,0,1,3,11,50,274,1764,13068,109584,\dots$$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. $k\;j$:s s.t. $\pi(j) > \pi(j+1)$, $k+1\;j$:s s.t. $\pi(j) \geq j$, $k\;j$:s s.t. $\pi(j) > j$.

$$E(n,k)=(n-k)E(n-1,k-1)+(k+1)E(n-1,k)$$
$$E(n,0)=E(n,n-1)=1$$
$$E(n,k)=\sum_{j=0}^k(-1)^j\binom{n+1}{j}(k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k)=S(n-1,k-1)+kS(n-1,k)$$
$$S(n,1)=S(n,n)=1$$
$$S(n,k)=\frac{1}{k!}\sum_{j=0}^k(-1)^{k-j}\binom{k}{j}j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n)=1,1,2,5,15,52,203,877,4140,21147,\dots$. For p prime,

$$B(p^m+n)\equiv mB(n)+B(n+1)\pmod{p}$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1n_2\cdots n_kn^{k-2}$
with degrees d_i : $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

6.3.7 Catalan numbers

$$C_n=\frac{1}{n+1}\binom{2n}{n}=\binom{2n}{n}-\binom{2n}{n+1}=\frac{(2n)!}{(n+1)!n!}$$
$$C_0=1,\; C_{n+1}=\frac{2(2n+1)}{n+2}C_n,\; C_{n+1}=\sum C_iC_{n-i}$$
$$C_n=1,1,2,5,14,42,132,429,1430,4862,16796,58786,\dots$$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).

- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Fundamentals

BellmanFord.h

Description: calculates shortest paths from s in a graph that might have negative edge weights. unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. assumes $V^2\max|w_i|<\sim 2^{63}$. Vertices are $1..n$ and the node vector has size $n+1$.
Time: $\mathcal{O}(VE)$

```
const ll INF = LLONG_MAX;
struct BellmanEdge { int a, b, w, s() { return a < b ? a : -a; } };
struct BellmanNode { ll dist = INF; int prev = -1; };

void bellman_ford(vector<BellmanNode>& nodes, vector<BellmanEdge>& eds, int s) {
    nodes[s].dist = 0;
    sort(begin(eds), end(eds), [](BellmanEdge a, BellmanEdge b) {
        return a.s() < b.s(); });

    int lim = (int)(nodes).size() / 2 + 2; // /3+100 with shuffled vertices
    for (int i = 0; i < lim; ++i) for (BellmanEdge edge : eds) {
        BellmanNode cur = nodes[edge.a], &dest = nodes[edge.b];
        if (abs(cur.dist) == INF) continue;
        ll d = cur.dist + edge.w;
        if (d < dest.dist) {
            dest.prev = edge.a;
            dest.dist = (i < lim-1 ? d : -INF);
        }
    }
    for (int i = 0; i < lim; ++i) for (BellmanEdge e : eds) {
        if (nodes[e.a].dist == -INF)
            nodes[e.b].dist = -INF;
    }
}
```

FloydWarshall.h

Description: calculates all-pairs shortest path in a directed graph that might have negative edge weights. input is a 1-indexed distance matrix m , where $m[i][j]$ = inf if i and j are not adjacent. as output, $m[i][j]$ is set to the shortest distance between i and j , inf if no path, or -inf if the path goes through a negative-weight cycle.
Time: $\mathcal{O}(N^3)$

```
const ll INF = 1LL << 62;
void floyd_warshall(vector<vector<ll>>& m) {
    int n = (int)m.size() - 1;
    for (int i = 1; i <= n; ++i) m[i][i] = min(m[i][i], 0LL);
    for (int k = 1; k <= n; ++k) for (int i = 1; i <= n; ++i) for (int j = 1; j <= n; ++j)
        if (m[i][k] != INF && m[k][j] != INF) {
            auto new_dist = max(m[i][k] + m[k][j], -INF);
            m[i][j] = min(m[i][j], new_dist);
        }
    for (int k = 1; k <= n; ++k) if (m[k][k] < 0) for (int i = 1; i <= n; ++i) for (int j = 1; j <= n; ++j)
        if (m[i][k] != INF && m[k][j] != INF) m[i][j] = -INF;
}
```

TopoSort.h

Description: topological sorting. given is an oriented graph. output is an ordering of vertices, such that there are edges only from left to right. if there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned. Vertices are numbered $1..n$ and the adjacency list has size $n+1$.
Time: $\mathcal{O}(|V|+|E|)$

```
vector<int> topo_sort(const vector<vector<int>>& gr) {
    vector<int> indeg((int)(gr).size()), q;
    for (int i = 1; i < (int)gr.size(); ++i) for (int x : gr[i])
        indeg[x]++;
    for (int i = 1; i < (int)gr.size(); ++i) if (indeg[i] == 0) q.push_back(i);
    for (int j = 0; j < ((int)(q).size()); ++j) for (int x : gr[q[j]])
        if (--indeg[x] == 0) q.push_back(x);
    return q;
}
```

7.2 Network flow

PushRelabel.h

Description: push-relabel using the highest label selection rule and the gap heuristic. quite fast in practice. to obtain the actual flow, look at positive values only. vertices are numbered $1..n$.
Time: $\mathcal{O}(V^2\sqrt{E})$

```
struct PushRelabel {
    struct Edge {
        int dest, back;
        ll f, c;
    };
    vector<vector<Edge>> g;
    vector<ll> ec;
    vector<Edge*> cur;
    vector<vector<int>> hs; vector<int> height;
    PushRelabel(int n) : g(n+1), ec(n+1), cur(n+1),
        hs(2*n+2), height(n+1) {}

    void add_edge(int s, int t, ll cap, ll rcap=0) {
        if (s == t) return;
        g[s].push_back({t, (int)(g[t]).size(), 0, cap});
        g[t].push_back({s, (int)(g[s]).size()-1, 0, rcap});
    }

    void add_flow(Edge& e, ll f) {
        Edge &back = g[e.dest][e.back];
        if (!ec[e.dest] && f) hs[height[e.dest]].push_back(e.dest);
        e.f += f; e.c -= f; ec[e.dest] += f;
        back.f -= f; back.c += f; ec[back.dest] -= f;
    }

    ll calc(int s, int t) {
        const int INF_HEIGHT = 1000000000;
        int n = (int)g.size() - 1; height[s] = n; ec[t] = 1;
        vector<int> co(2*n+2); co[0] = n-1;
        for (int i = 1; i <= n; ++i) cur[i] = g[i].data();
        for (Edge& e : g[s]) add_flow(e, e.c);

        for (int hi = 0;;) {
            while (hs[hi].empty()) if (!hi--) return -ec[s];
            int u = hs[hi].back(); hs[hi].pop_back();
            while (ec[u] > 0) // discharge u
                if (cur[u] == g[u].data() + (int)(g[u]).size()) {
                    height[u] = INF_HEIGHT;
                    for (Edge& e : g[u]) if (e.c && height[u] > height[e.dest]+1)
                        height[u] = height[e.dest]+1, cur[u] = &e;
                    if (++co[height[u]],!--co[hi] && hi < n)
```

```
        for (int i = 1; i <= n; ++i) if (hi < height[i] && height[i] < n)
            --co[height[i]], height[i] = n + 1;
        hi = height[u];
    } else if (cur[u]->c && height[u] == height[cur[u]->dest]+1)
        add_flow(*cur[u], min(ec[u], cur[u]->c));
    else ++cur[u];
}
}
bool left_of_min_cut(int a) { return height[a] >= (int)g.size() - 1; }
};
```

MinCostMaxFlow.h

Description: min-cost max-flow. if costs can be negative, call set_pi before max_flow, but note that negative cost cycles are not supported. to obtain the actual flow, look at positive values only. vertices are numbered 1..n.
Time: $\mathcal{O}(FE \log(V))$ where F is max flow. $\mathcal{O}(VE)$ for set_pi.

```
const ll INF = numeric_limits<ll>::max() / 4;

struct MinCostMaxFlowGraph {
    struct Edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int vertex_count;
    vector<vector<Edge*>> ed;
    vector<int> seen;
    vector<ll> dist, pi;
    vector<Edge*> par;

    MinCostMaxFlowGraph(int n) : vertex_count(n), ed(n + 1), seen(n + 1),
        dist(n + 1), pi(n + 1), par(n + 1) {}

    void add_edge(int from, int to, ll cap, ll cost) {
        if (from == to) return;
        ed[from].push_back(Edge{ from,to,(int)(ed[to]).size(),cap,
            cost,0 });
        ed[to].push_back(Edge{ to,from,(int)(ed[from]).size()-1,0,-
            cost,0 });
    }

    void shortest_path(int s) {
        fill(begin(seen), end(seen), 0);
        fill(begin(dist), end(dist), INF);
        fill(begin(par), end(par), nullptr);
        dist[s] = 0; ll di;

        priority_queue<pair<ll, int>, vector<pair<ll, int>>,
            greater<pair<ll, int>>> q;
        q.push({ 0, s });

        while (!q.empty()) {
            s = q.top().second; q.pop();
            if (seen[s]) continue;
            seen[s] = 1; di = dist[s] + pi[s];
            for (Edge& e : ed[s]) if (!seen[e.to]) {
                ll val = di - pi[e.to] + e.cost;
                if (e.cap - e.flow > 0 && val < dist[e.to]) {
                    dist[e.to] = val;
                    par[e.to] = &e;
                    q.push({dist[e.to], e.to});
                }
            }
        }
    }
};
```

```
        for (int i = 1; i <= vertex_count; ++i) pi[i] = min(pi[i] + dist[i], INF);
    }

    pair<ll, ll> max_flow(int s, int t) {
        ll total_flow = 0, total_cost = 0;
        while (shortest_path(s), seen[t]) {
            ll fl = INF;
            for (Edge* x = par[t]; x; x = par[x->from])
                fl = min(fl, x->cap - x->flow);

            total_flow += fl;
            for (Edge* x = par[t]; x; x = par[x->from]) {
                x->flow += fl;
                ed[x->to][x->rev].flow -= fl;
            }
        }
        for (int i = 1; i <= vertex_count; ++i)
            for (Edge& e : ed[i]) total_cost += e.cost * e.flow;
        return {total_flow, total_cost/2};
    }

    // if some costs can be negative, call this before max_flow:
    void set_pi(int s) { // (otherwise, leave this out)
        fill(begin(pi), end(pi), INF); pi[s] = 0;
        int it = vertex_count, ch = 1; ll v;
        while (ch-- && it--)
            for (int i = 1; i <= vertex_count; ++i) if (pi[i] != INF)
                for (Edge& e : ed[i]) if (e.cap)
                    if ((v = pi[i] + e.cost) < pi[e.to])
                        pi[e.to] = v, ch = 1;
        assert(it >= 0); // negative cost cycle
    }
};
```

EdmondsKarp.h

Description: flow algorithm with guaranteed complexity $\mathcal{O}(VE^2)$. to get edge flow values, compare capacities before and after, and take the positive values only.

```
template<class T> T edmonds_karp(vector<unordered_map<int, T>&&
    graph, int source, int sink) {
    assert(source != sink);
    T flow = 0;
    vector<int> par((int)(graph).size()), q = par;

    for (;;) {
        fill(begin(par), end(par), -1);
        par[source] = 0;
        int ptr = 1;
        q[0] = source;

        for (int i = 0; i < ptr; ++i) {
            int x = q[i];
            for (auto e : graph[x]) {
                if (par[e.first] == -1 && e.second > 0) {
                    par[e.first] = x;
                    q[ptr++] = e.first;
                    if (e.first == sink) goto out;
                }
            }
        }
        return flow;
    out:
        T inc = numeric_limits<T>::max();
        for (int y = sink; y != source; y = par[y])
            inc = min(inc, graph[par[y]][y]);

        flow += inc;
    }
};
```

```
        for (int y = sink; y != source; y = par[y]) {
            int p = par[y];
            if ((graph[p][y] -= inc) <= 0) graph[p].erase(y);
            graph[y][p] += inc;
        }
    }
};
```

MinCut.h

Description: after running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s , only traversing edges with positive residual capacity.

GlobalMinCut.h

Description: find a global minimum cut in an undirected graph represented by a 1-indexed adjacency matrix. Returned vertices are numbered from 1.
Time: $\mathcal{O}(V^3)$

```
pair<int, vector<int>> global_min_cut(const vector<vector<int>
    >>& graph) {
    pair<int, vector<int>> best = {INT_MAX, {}};
    int n = (int)graph.size() - 1;
    vector<vector<int>> mat(n, vector<int>(n));
    for (int i = 0; i < n; ++i) for (int j = 0; j < n; ++j)
        mat[i][j] = graph[i+1][j+1];
    vector<vector<int>> co(n);
    for (int i = 0; i < n; ++i) co[i] = {i + 1};
    for (int ph = 1; ph < (n); ++ph) {
        vector<int> w = mat[0];
        size_t s = 0, t = 0;
        for (int it = 0; it < (n-ph); ++it) { //  $\mathcal{O}(V^2) \rightarrow \mathcal{O}(E \log V)$  with prio. queue
            w[t] = INT_MIN;
            s = t, t = max_element(begin(w), end(w)) - w.begin();
            for (int i = 0; i < (n); ++i) w[i] += mat[t][i];
        }
        best = min(best, {w[t] - mat[t][t], co[t]});
        co[s].insert(co[s].end(), begin(co[t]), end(co[t]));
        for (int i = 0; i < (n); ++i) mat[s][i] += mat[t][i];
        for (int i = 0; i < (n); ++i) mat[i][s] = mat[s][i];
        mat[0][t] = INT_MIN;
    }
    return best;
};
```

GomoryHu.h

Description: given a list of edges representing an undirected flow graph, returns edges of the gomory-hu tree. the max flow between any pair of vertices is given by minimum array<ll, 3> weight along the gomory-hu tree path.
Time: $\mathcal{O}(V)$ flow computations

```
"PushRelabel.h"
vector<array<ll, 3>> gomory_hu(int n, vector<array<ll, 3>>
    edges) {
    vector<array<ll, 3>> tree;
    vector<int> parent(n + 1, 1);
    for (int i = 2; i <= n; ++i) {
        PushRelabel flow(n); // Dinic also works
        for (array<ll, 3> edge : edges)
            flow.add_edge((int)edge[0], (int)edge[1], edge[2], edge[2]);
        tree.push_back({i, parent[i], flow.calc(i, parent[i])});
        for (int j = i + 1; j <= n; ++j)
            if (parent[j] == parent[i] && flow.left_of_min_cut(j))
                parent[j] = i;
    }
    return tree;
};
```

7.3 Matching

HopcroftKarp.h

Description: fast bipartite matching algorithm. graph g should be a list of neighbors of the left partition, and r should be a vector full of -1 's of the same size as the right partition. returns the size of the matching. $r[i]$ will be the match for vertex i on the right side, or -1 if it's not matched. both partitions are numbered from 1; vectors include an unused index 0.

Time: $\mathcal{O}\left(E\sqrt{V}\right)$

```
int hopcroft_karp(vector<vector<int>>& g, vector<int>& r) {
    int n = (int)g.size() - 1, res = 0;
    vector<int> l(n + 1, -1), q(n + 1), d(n + 1);
    auto dfs = [&](auto f, int u) -> bool {
        int t = exchange(d[u], 0) + 1;
        for (int v : g[u])
            if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
                return l[u] = v, r[v] = u, 1;
        return 0;
    };
    for (int t = 0, f = 0;; t = f = 0, d.assign(n + 1, 0)) {
        for (int i = 1; i <= n; ++i) if (l[i] == -1) q[t++] = i, d[i] = 1;
        for (int i = 0; i < (t); ++i) for (int v : g[q[i]]) {
            if (r[v] == -1) f = 1;
            else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
        }
        if (!f) return res;
        for (int i = 1; i <= n; ++i) if (l[i] == -1) res += dfs(dfs, i);
    }
}
```

DFSMatching.h

Description: simple bipartite matching algorithm. graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1 's of the same size as the right partition. returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched. both partitions are numbered from 1; vectors include an unused index 0.

Usage: vector<int> btoa(m + 1, -1); dfs_matching(g, btoa);
Time: $\mathcal{O}(VE)$

```
bool find_augmenting_path(int j, vector<vector<int>>& g, vector<int>& btoa,
    vector<int>& vis) {
    if (btoa[j] == -1) return 1;
    vis[j] = 1; int di = btoa[j];
    for (int e : g[di])
        if (!vis[e] && find_augmenting_path(e, g, btoa, vis)) {
            btoa[e] = di;
            return 1;
        }
    return 0;
}
int dfs_matching(vector<vector<int>>& g, vector<int>& btoa) {
    vector<int> vis;
    for (int i = 1; i < (int)g.size(); ++i) {
        vis.assign((int)btoa.size(), 0);
        for (int j : g[i])
            if (find_augmenting_path(j, g, btoa, vis)) {
                btoa[j] = i;
                break;
            }
    }
    return (int)btoa.size() - 1
        - (int)count(next(begin(btoa)), end(btoa), -1);
}
```

MinimumVertexCover.h

Description: finds a minimum vertex cover in a bipartite graph. the size is the same as the size of a maximum matching, and the complement is a maximum independent set. left vertices are $1..n$, right vertices are $1..m$ in the graph, and returned right vertices are shifted to $n + 1..n + m$.

```
"DFSMatching.h"
vector<int> cover(vector<vector<int>>& g, int n, int m) {
    vector<int> match(m + 1, -1);
    int res = dfs_matching(g, match);
    vector<bool> lfound(n + 1, true), seen(m + 1);
    for (int i = 1; i <= m; ++i) if (match[i] != -1) lfound[match[i]] = false;
    vector<int> q, cover;
    for (int i = 1; i <= n; ++i) if (lfound[i]) q.push_back(i);
    while (!q.empty()) {
        int i = q.back(); q.pop_back();
        lfound[i] = 1;
        for (int e : g[i]) if (!seen[e] && match[e] != -1) {
            seen[e] = true;
            q.push_back(match[e]);
        }
    }
    for (int i = 1; i <= n; ++i) if (!lfound[i]) cover.push_back(i);
    for (int i = 1; i <= m; ++i) if (seen[i]) cover.push_back(n+i);
    assert((int)(cover).size() == res);
    return cover;
}
```

WeightedMatching.h

Description: given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. takes a 1-indexed cost matrix, where $cost[i][j]$ is the cost for $L[i]$ to be matched with $R[j]$ and returns (min cost, match), where $L[i]$ is matched with $R[match[i]]$. negate costs for max cost. requires $N \leq M$.

Time: $\mathcal{O}(N^2M)$

```
pair<int, vector<int>> hungarian(const vector<vector<int>>& cost) {
    if (cost.size() <= 1) return {0, vector<int>(1)};
    int n = (int)cost.size() - 1, m = (int)cost[1].size() - 1;
    vector<int> u(n + 1), v(m + 1), p(m + 1), answer(n + 1);
    for (int i = 1; i <= n; ++i) {
        p[0] = i;
        int j0 = 0; // add "dummy" worker 0
        vector<int> dist(m + 1, INT_MAX), pre(m + 1, -1);
        vector<bool> done(m + 1);
        do { // dijkstra
            done[j0] = true;
            int i0 = p[j0], j1, delta = INT_MAX;
            for (int j = 1; j <= m; ++j) if (!done[j]) {
                auto cur = cost[i0][j] - u[i0] - v[j];
                if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
                if (dist[j] < delta) delta = dist[j], j1 = j;
            }
            for (int j = 0; j <= m; ++j) {
                if (done[j]) u[p[j]] += delta, v[j] -= delta;
                else dist[j] -= delta;
            }
            j0 = j1;
        } while (p[j0]);
        while (j0) { // update alternating path
            int j1 = pre[j0];
            p[j0] = p[j1], j0 = j1;
        }
    }
    for (int j = 1; j <= m; ++j) if (p[j]) answer[p[j]] = j;
}
```

```
return {-v[0], answer}; // min cost
}
```

GeneralMatching.h

Description: matching for general graphs. fails with probability N/MOD .
Time: $\mathcal{O}(N^3)$

```
"../numerical/MatrixInverse-mod.h"
a850a6, 41 lines
vector<pii> general_matching(int n, vector<pii>& edges) {
    vector<vector<ll>> mat(n + 1, vector<ll>(n + 1)), inverse;
    for (pii pa : edges) {
        int a = pa.first, b = pa.second, r = rand() % MOD;
        mat[a][b] = r, mat[b][a] = (MOD - r) % MOD;
    }

    int r = mat_inv(inverse = mat), size = 2*n - r, fi, fj;
    assert(r % 2 == 0);

    if (size != n) do {
        mat.resize(size + 1, vector<ll>(size + 1));
        for (int i = 1; i <= n; ++i) {
            mat[i].resize(size + 1);
            for (int j = n + 1; j <= size; ++j) {
                int r = rand() % MOD;
                mat[i][j] = r, mat[j][i] = (MOD - r) % MOD;
            }
        }
    } while (mat_inv(inverse = mat) != size);

    vector<int> has(size + 1, 1); vector<pii> ret;
    for (int it = 0; it < size/2; ++it) {
        for (int i = 1; i <= size; ++i) if (has[i])
            for (int j = i+1; j <= size; ++j) if (inverse[i][j] && mat[i][j]) {
                fi = i; fj = j; goto done;
            }
        assert(0); done:
        if (fj <= n) ret.emplace_back(fi, fj);
        has[fi] = has[fj] = 0;
        for (int sw = 0; sw < (2); ++sw) {
            ll a = mod_pow(inverse[fi][fj], MOD - 2);
            for (int i = 1; i <= size; ++i) if (has[i] && inverse[i][fj]) {
                ll b = inverse[i][fj] * a % MOD;
                for (int j = 1; j <= size; ++j)
                    inverse[i][j] = (inverse[i][j] - inverse[fi][j] * b) % MOD;
            }
            swap(fi, fj);
        }
    }
    return ret;
}
```

7.4 DFS algorithms

SCC.h

Description: finds strongly connected components in a directed graph. if vertices u, v belong to the same component, we can reach u from v and vice versa.

Usage: scc(graph, [&](vector<int>& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). Vertices and component indices are numbered from 1; the graph and output vectors have an unused index 0.

Time: $\mathcal{O}(E + V)$

```
vector<int> val, comp, z, cont;
int dfs_time, component_count;
```

```

template<class G, class F> int scc_dfs(int j, G& g, F& f) {
    int low = val[j] = ++dfs_time, x; z.push_back(j);
    for (auto e : g[j]) if (comp[e] < 0)
        low = min(low, val[e] ? val[e] : scc_dfs(e, g, f));

    if (low == val[j]) {
        ++component_count;
        do {
            x = z.back(); z.pop_back();
            comp[x] = component_count;
            cont.push_back(x);
        } while (x != j);
        f(cont); cont.clear();
    }
    return val[j] = low;
}

template<class G, class F> void scc(G& g, F f) {
    int n = (int)g.size() - 1;
    val.assign(n + 1, 0); comp.assign(n + 1, -1);
    dfs_time = component_count = 0;
    for (int i = 1; i <= n; ++i) if (comp[i] < 0) scc_dfs(i, g, f
);
}

```

BiconnectedComponents.h

Description: finds all biconnected components in an undirected graph, and runs a callback for the edges in each. in a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). note that a node can be in several components. an edge which is not in a component is a bridge, i.e., not part of any cycle.

Usage: int edge_id = 1; ed.resize(n + 1);
for each edge (a,b) {
ed[a].emplace_back(b, edge_id);
ed[b].emplace_back(a, edge_id++); }
biconnected_components([&](const vector<int>& edges) {...});
vertices and edge IDs are numbered from 1.

Time: $\mathcal{O}(E + V)$

538eb0, 34 lines

```

vector<int> num, st;
vector<vector<pii>> ed;
int dfs_time;
template<class F>
int biconnected_dfs(int at, int parent_edge, F& f) {
    int me = num[at] = ++dfs_time, top = me;
    for (auto [y, edge_id] : ed[at]) if (edge_id != parent_edge)
        {
            if (num[y]) {
                top = min(top, num[y]);
                if (num[y] < me)
                    st.push_back(edge_id);
            } else {
                int stack_size = (int)st.size();
                int up = biconnected_dfs(y, edge_id, f);
                top = min(top, up);
                if (up == me) {
                    st.push_back(edge_id);
                    f(vector<int>(st.begin() + stack_size, st.end()));
                    st.resize(stack_size);
                }
                else if (up < me) st.push_back(edge_id);
                else { /* edge_id is a bridge */ }
            }
        }
    return top;
}

template<class F>
void biconnected_components(F f) {
    dfs_time = 0;

```

```

    num.assign((int)ed.size(), 0);
    for (int i = 1; i < (int)ed.size(); ++i)
        if (!num[i]) biconnected_dfs(i, 0, f);
}

2sat.h
Description: calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type (a||b)&&(a||c)&&(d||b)&&... becomes true, or reports that it is unsatisfiable. variables are numbered 1..n and negation is represented by a minus sign.
Usage: TwoSat ts(number of boolean variables);
ts.either(1, -4); // var 1 is true or var 4 is false
ts.set.value(3); // var 3 is true
ts.at_most_one({1,-2,3}); // <= 1 of these literals is true
ts.solve(); // returns true iff it is solvable
ts.values[1..n] holds the assigned values to the vars
Time:  $\mathcal{O}(N + E)$ , where N is the number of boolean variables, and E is the number of clauses.

```

670862, 61 lines

```

struct TwoSat {
    int variable_count;
    vector<vector<int>>> gr;
    vector<int> values; // 0 = false, 1 = true

    TwoSat(int n = 0) : variable_count(n), gr(2*n) {}

    int add_var() { // (optional)
        gr.emplace_back();
        gr.emplace_back();
        return ++variable_count;
    }

    int literal_node(int literal) {
        assert(literal != 0);
        return 2 * (abs(literal) - 1) + (literal < 0);
    }

    void either(int f, int j) {
        f = literal_node(f);
        j = literal_node(j);
        gr[f].push_back(j^1);
        gr[j].push_back(f^1);
    }

    void set_value(int x) { either(x, x); }

    void at_most_one(const vector<int>& li) { // (optional)
        if ((int)(li).size() <= 1) return;
        int cur = -li[0];
        for (int i = 2; i < ((int)(li).size()); ++i) {
            int next = add_var();
            either(cur, -li[i]);
            either(cur, next);
            either(-li[i], next);
            cur = -next;
        }
        either(cur, -li[1]);
    }

    vector<int> val, comp, z; int time = 0;
    int dfs(int i) {
        int low = val[i] = ++time, x; z.push_back(i);
        for(int e : gr[i]) if (!comp[e])
            low = min(low, val[e] ? dfs(e));
        if (low == val[i]) do {
            x = z.back(); z.pop_back();
            comp[x] = low;
            if (values[(x >> 1) + 1] == -1)
                values[(x >> 1) + 1] = x & 1;
        } while (x != i);

```

```

        return val[i] = low;
    }

    bool solve() {
        values.assign(variable_count + 1, -1);
        val.assign(2*variable_count, 0); comp = val;
        for (int i = 0; i < 2*variable_count; ++i) if (!comp[i])
            dfs(i);
        for (int i = 0; i < variable_count; ++i)
            if (comp[2*i] == comp[2*i+1]) return 0;
        return 1;
    }
};

```

EulerWalk.h

Description: eulerian undirected/directed path/cycle algorithm. input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. returns a list of nodes in the eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. vertices and global edge indices are numbered from 1; the adjacency list has size n + 1. to get edge indices back, add .second to s and ret.

Time: $\mathcal{O}(V + E)$

c86cb3, 16 lines

```

vector<int> euler_walk(vector<vector<pii>>& gr, int edge_count,
    int src=1) {
    int n = (int)(gr).size();
    vector<int> degree_delta(n), its(n), used(edge_count + 1),
        ret, s = {src};
    degree_delta[src]++; // to allow euler paths, not just cycles
    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x], end = (int)(gr[x]).
            size();
        if (it == end){ ret.push_back(x); s.pop_back(); continue; }
        tie(y, e) = gr[x][it++];
        if (!used[e]) {
            degree_delta[x]--, degree_delta[y]++;
            used[e] = 1; s.push_back(y);
        }
    }
    for (int i = 1; i < n; ++i)
        if (degree_delta[i] < 0 || (int)ret.size() != edge_count+1)
            return {};
    return {ret.rbegin(), ret.rend()};
}

```

7.5 Coloring

EdgeColoring.h

Description: given a simple, undirected graph with max degree D , computes a $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.) vertices and returned edge colors are numbered from 1. The result has an unused index 0, so result[i] is the color of edges[i-1].

Time: $\mathcal{O}(NM)$

046be1, 34 lines

```

vector<int> edge_coloring(int n, vector<pii> eds) {
    vector<int> cc(n + 2), ret(eds.size() + 1), fan(n + 1), free(
        n + 1), loc;
    for (pii e : eds) ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(begin(cc), end(cc)) + 1;
    vector<vector<int>> adj(n + 1, vector<int>(ncols, -1));
    for (pii e : eds) {
        tie(u, v) = e;
        fan[0] = v;
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind = 0, i = 0;
        while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
            loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
        cc[loc[d]] = c;

```

```
for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
    swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
while (adj[fan[i]][d] != -1) {
    int left = fan[i], right = fan[++i], e = cc[i];
    adj[u][e] = left;
    adj[left][e] = u;
    adj[right][e] = -1;
    free[right] = e;
}
adj[u][d] = fan[i];
adj[fan[i]][d] = u;
for (int y : {fan[0], u, end})
    for (int& z = free[y] = 0; adj[y][z] != -1; z++);
}
for (int i = 1; i <= (int)eds.size(); ++i) {
    int& color = ret[i];
    for (tie(u, v) = eds[i-1]; adj[u][color] != v;) ++color;
    ++color;
}
return ret;
}
```

7.6 Heuristics

MaximalCliques.h

Description: runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). callback is given a bitset representing the maximal clique. vertices are numbered 1..n and the matrix has size $n + 1$.

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

c4cc42, 19 lines

```
template<class F>
void cliques_rec(vector<bitset<128>>& eds, F& f, bitset<128>
    possible,
    bitset<128> excluded, bitset<128> current) {
    if (!possible.any()) { if (!excluded.any()) f(current);
        return; }
    auto q = (possible | excluded)._Find_first();
    auto candidates = possible & ~eds[q];
    for (int i = 1; i < (int)eds.size(); ++i) if (candidates[i]) {
        current[i] = 1;
        cliques_rec(eds, f, possible & eds[i], excluded & eds[i],
            current);
        current[i] = possible[i] = 0; excluded[i] = 1;
    }
}
```

```
template<class F>
void cliques(vector<bitset<128>>& eds, F f) {
    bitset<128> possible;
    for (int i = 1; i < (int)eds.size(); ++i) possible[i] = 1;
    cliques_rec(eds, f, possible, {}, {});
}
```

MaximumClique.h

Description: quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). can be used to find a maximum independent set by finding a clique of the complement graph. vertices are numbered 1..n and the matrix has size $n + 1$.

Time: runs in about 1s for n=155 and worst case random graphs (p=.90). runs faster for sparse graphs.

3ef4ba, 48 lines

```
struct MaximumClique {
    double limit=0.025, pk=0;
    struct Vertex { int i, d=0; };
    vector<bitset<200>> e;
    vector<Vertex> vertices;
    vector<vector<int>> colors;
```

```
vector<int> maximum, current, steps, old_steps;
void init(vector<Vertex>& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
    sort(begin(r), end(r), [](auto a, auto b) { return a.d > b.d; });
    int mx_d = r[0].d;
    for (int i = 0; i < ((int)(r).size()); ++i) r[i].d = min(i,
        mx_d) + 1;
}
void expand(vector<Vertex>& remaining, int level = 1) {
    steps[level] += steps[level - 1] - old_steps[level];
    old_steps[level] = steps[level - 1];
    while (!remaining.empty()) {
        if ((int)current.size() + remaining.back().d <= (int)
            maximum.size()) return;
        current.push_back(remaining.back().i);
        vector<Vertex> next_vertices;
        for(auto v:remaining) if (e[remaining.back().i][v.i])
            next_vertices.push_back({v.i});
        if (!next_vertices.empty()) {
            if (steps[level]++ / ++pk < limit) init(next_vertices);
            int j = 0, mxk = 1, mnk = max((int)maximum.size() - (
                int)current.size() + 1, 1);
            colors[1].clear(), colors[2].clear();
            for (auto v : next_vertices) {
                int k = 1;
                auto f = [&](int i) { return e[v.i][i]; };
                while (any_of(begin(colors[k]), end(colors[k]), f)) k
                    ++;
                if (k > mxk) mxk = k, colors[mxk + 1].clear();
                if (k < mnk) next_vertices[j++] .i = v.i;
                colors[k].push_back(v.i);
            }
            if (j > 0) next_vertices[j - 1].d = 0;
            for (int k = mnk; k <= mxk; ++k) for (int i : colors[k]
                )
                next_vertices[j].i = i, next_vertices[j++].d = k;
            expand(next_vertices, level + 1);
        } else if (current.size() > maximum.size()) maximum =
            current;
        current.pop_back(), remaining.pop_back();
    }
}
vector<int> max_clique() { init(vertices), expand(vertices);
    return maximum; }
MaximumClique(vector<bitset<200>> conn) : e(conn), colors(e.
    size() + 1),
    steps(colors.size()), old_steps(steps) {
    for (int i = 1; i < (int)e.size(); ++i) vertices.push_back(
        {i});
}
```

MaximumIndependentSet.h

Description: to obtain a maximum independent set of a graph, find a max clique of the complement. if the graph is bipartite, see minimum_vertex_cover.

7.7 Trees

BinaryLifting.h

Description: calculate power of two jumps in a tree, to support fast upward jumps and LCAs. vertices are numbered 1..n, the parent vector has size $n + 1$, and the root points to itself.

Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

781c85, 25 lines

```
vector<vector<int>> tree_jump(const vector<int>& parent) {
    int on = 1, d = 1;
```

```
while (on < (int)parent.size() - 1) on *= 2, d++;
vector<vector<int>> jump(d, parent);
for (int i = 1; i < d; ++i) for (int j = 1; j < (int)parent.
    size(); ++j)
    jump[i][j] = jump[i-1][jump[i-1][j]];
return jump;
}
```

```
int jump(const vector<vector<int>>& table, int node, int steps)
{
    for (int i = 0; i < (int)table.size(); ++i)
        if (steps & (1 << i)) node = table[i][node];
    return node;
}
```

```
int lca(const vector<vector<int>>& table, const vector<int>&
    depth, int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    a = jump(table, a, depth[a] - depth[b]);
    if (a == b) return a;
    for (int i = (int)table.size(); i--;) {
        int c = table[i][a], d = table[i][b];
        if (c != d) a = c, b = d;
    }
    return table[0][a];
}
```

LCA.h

Description: data structure for computing lowest common ancestors in a tree (with 1 as root). graph should have size $n + 1$ and use vertices 1..n. or undirected.

Time: $\mathcal{O}(N \log N + Q)$

../data-structures/RMQ.h" f89e02, 28 lines

```
struct LcaQuery {
private:
    int timer = 0;
    vector<int> time, path = vector<int>(1), ret = vector<int>(1)
        ;
    RangeMinimumQuery<int> range_minimum;

    void dfs(vector<vector<int>>& graph, int v, int par) {
        time[v] = ++timer;
        for (int y : graph[v]) if (y != par) {
            path.push_back(v), ret.push_back(time[v]);
            dfs(graph, y, v);
        }
    }
}
```

```
public:
    LcaQuery(vector<vector<int>>& graph) : time((int)graph.size()
        ),
        range_minimum((dfs(graph, 1, 0), ret)) {}
    int size() const { return (int)time.size() - 1; }
    int entry_time(int vertex) const { return time[vertex]; }
    int query(int a, int b) {
        if (a == b) return a;
        tie(a, b) = minmax(time[a], time[b]);
        return path[range_minimum.query(a, b - 1)];
    }
    // dist(a,b){return depth[a] + depth[b] - 2*depth[query(a,b)
        ];}
};
```

CompressTree.h

Description: given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise lca's and compressing edges. returns a 1-indexed list of (parent, original_index), with index 0 unused. the root is at index 1 and points to itself.

Time: $\mathcal{O}(|S| \log |S|)$

"LCA.h" 556b12, 24 lines

```
vector<pair<int, int>> compress_tree(LcaQuery& lca, const
    vector<int>& subset) {
    if (subset.empty()) return vector<pair<int, int>>(1);
    static vector<int> reverse_index;
    reverse_index.resize(lca.size() + 1);
    vector<int> vertices = subset;
    auto cmp = [&](int a, int b) { return lca.entry_time(a) < lca
        .entry_time(b); };
    sort(begin(vertices), end(vertices), cmp);
    int m = (int)vertices.size() - 1;
    for (int i = 0; i < m; ++i) {
        int a = vertices[i], b = vertices[i + 1];
        vertices.push_back(lca.query(a, b));
    }
    sort(begin(vertices), end(vertices), cmp);
    vertices.erase(unique(begin(vertices), end(vertices)), end(
        vertices));
    for (int i = 0; i < (int)vertices.size(); ++i)
        reverse_index[vertices[i]] = i + 1;
    vector<pair<int, int>> result(1);
    result.emplace_back(1, vertices[0]);
    for (int i = 0; i + 1 < (int)vertices.size(); ++i) {
        int a = vertices[i], b = vertices[i + 1];
        result.emplace_back(reverse_index[lca.query(a, b)], b);
    }
    return result;
}
```

HLD.h

Description: decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most $\log(n)$ light edges. code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. all values initialized to the segtree default. vertices are $1..n$, root is 1, and the adjacency list has size $n + 1$.

Time: $\mathcal{O}((\log N)^2)$

"../data-structures/LazySegmentTree.h" 0139c9, 50 lines

```
template <bool VALUES_ON_EDGES> struct HeavyLightDecomposition
{
    int vertex_count, timer = 1;
    vector<vector<int>> adj;
    vector<int> par, siz, rt, pos;
    LazySegmentTree *tree;
    HeavyLightDecomposition(vector<vector<int>> adj_)
        : vertex_count((int)adj_.size() - 1), adj(adj_), par(adj_.
            size(), -1),
          siz(adj_.size(), 1), rt(adj_.size(), pos(adj_.size(),
            tree(new LazySegmentTree(1, vertex_count)) {
                rt[1] = 1; dfs_sz(1); dfs_hld(1);
            }
        )
    {
    void dfs_sz(int v) {
        for (int& u : adj[v]) {
            adj[u].erase(find(begin(adj[u]), end(adj[u]), v));
            par[u] = v;
            dfs_sz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
}
```

```
void dfs_hld(int v) {
    pos[v] = timer++;
    for (int u : adj[v]) {
        rt[u] = (u == adj[v][0] ? rt[v] : u);
        dfs_hld(u);
    }
}

template <class B> void process(int u, int v, B op) {
    for (; v = par[rt[v]]) {
        if (pos[u] > pos[v]) swap(u, v);
        if (rt[u] == rt[v]) break;
        op(pos[rt[v]], pos[v]);
    }
    if (pos[u] + VALUES_ON_EDGES <= pos[v])
        op(pos[u] + VALUES_ON_EDGES, pos[v]);
}

void modify_path(int u, int v, int val) {
    process(u, v, [&](int l, int r) { tree->add(l, r, val); });
}

int query_path(int u, int v) { // modify depending on problem
    int res = -1e9;
    process(u, v, [&](int l, int r) {
        res = max(res, tree->query(l, r));
    });
    return res;
}

int query_subtree(int v) { // modify_subtree is similar
    return tree->query(pos[v] + VALUES_ON_EDGES, pos[v] + siz[v]
        - 1);
}
};
```

LinkCutTree.h

Description: represents a forest of unrooted trees. you can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree. vertices are numbered $1..n$.

Time: all operations take amortized $\mathcal{O}(\log N)$.

2b813c, 90 lines

```
struct LinkCutNode { // splay tree. root's pp contains tree's
    parent.
    LinkCutNode *p = 0, *pp = 0, *c[2];
    bool flip = 0;
    LinkCutNode() { c[0] = c[1] = 0; fix(); }
    void fix() {
        if (c[0]) c[0]->p = this;
        if (c[1]) c[1]->p = this;
        // (+ update sum of subtree elements etc. if wanted)
    }
    void push_flip() {
        if (!flip) return;
        flip = 0; swap(c[0], c[1]);
        if (c[0]) c[0]->flip ^= 1;
        if (c[1]) c[1]->flip ^= 1;
    }
    int up() { return p ? p->c[1] == this : -1; }
    void rot(int i, int b) {
        int h = i ^ b;
        LinkCutNode *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ?
            y : x;
        if ((y->p = p)) p->c[up()] = y;
        c[i] = z->c[i ^ 1];
        if (b < 2) {
            x->c[h] = y->c[h ^ 1];
            y->c[h ^ 1] = x;
        }
        z->c[i ^ 1] = this;
        fix(); x->fix(); y->fix();
        if (p) p->fix();
        swap(pp, y->pp);
    }
}
```

```
}
void splay() {
    for (push_flip(); p; ) {
        if (p->p) p->p->push_flip();
        p->push_flip(); push_flip();
        int c1 = up(), c2 = p->up();
        if (c2 == -1) p->rot(c1, 2);
        else p->p->rot(c2, c1 != c2);
    }
}
LinkCutNode* first() {
    push_flip();
    return c[0] ? c[0]->first() : (splay(), this);
}
};
```

```
struct LinkCutTree {
    vector<LinkCutNode> nodes;
    LinkCutTree(int n) : nodes(n + 1) {}

    void link(int u, int v) { // add an edge (u, v)
        assert(!connected(u, v));
        make_root(&nodes[u]);
        nodes[u].pp = &nodes[v];
    }
    void cut(int u, int v) { // remove an edge (u, v)
        LinkCutNode *x = &nodes[u], *top = &nodes[v];
        make_root(top); x->splay();
        assert(top == (x->pp ? x->c[0]));
        if (x->pp) x->pp = 0;
        else {
            x->c[0] = top->p = 0;
            x->fix();
        }
    }
    bool connected(int u, int v) { // are u, v in the same tree?
        LinkCutNode* nu = access(&nodes[u])->first();
        return nu == access(&nodes[v])->first();
    }
    void make_root(LinkCutNode* u) {
        access(u);
        u->splay();
        if (u->c[0]) {
            u->c[0]->p = 0;
            u->c[0]->flip ^= 1;
            u->c[0]->pp = u;
            u->c[0] = 0;
            u->fix();
        }
    }
    LinkCutNode* access(LinkCutNode* u) {
        u->splay();
        while (LinkCutNode* pp = u->pp) {
            pp->splay(); u->pp = 0;
            if (pp->c[1]) {
                pp->c[1]->p = 0; pp->c[1]->pp = pp;
            }
            pp->c[1] = u; pp->fix(); u = pp;
        }
        return u;
    }
};
```

DirectedMST.h

Description: finds a minimum spanning tree/arborescence of a directed graph. vertices are numbered $1..n$; the returned parent vector also uses indices $1..n$. if no MST exists, returns -1.

Time: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" 3e7c80, 60 lines

```
struct Edge { int a, b; ll w; };
```

```

struct HeapNode {
    Edge key;
    HeapNode *l, *r;
    ll delta;
    void prop() {
        key.w += delta;
        if (l) l->delta += delta;
        if (r) r->delta += delta;
        delta = 0;
    }
    Edge top() { prop(); return key; }
};

HeapNode *merge(HeapNode *a, HeapNode *b) {
    if (!a || !b) return a ? b;
    a->prop(), b->prop();
    if (a->key.w > b->key.w) swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}

void pop(HeapNode& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vector<int>> dmst(int n, int r, vector<Edge>& g) {
    RollbackUnionFind uf(n);
    vector<HeapNode*> heap(n + 1);
    for (Edge e : g) heap[e.b] = merge(heap[e.b], new HeapNode(e));
    ll res = 0;
    vector<int> seen(n + 1, -1), path(n + 1), par(n + 1);
    seen[r] = r;
    vector<Edge> chosen(n + 1), in(n + 1, {-1, -1}), comp;
    deque<tuple<int, int, vector<Edge>>> cycs;
    for (int s = 1; s <= n; ++s) {
        int u = s, qi = 0, w;
        while (seen[u] < 0) {
            if (!heap[u]) return {-1, {}};
            Edge e = heap[u]->top();
            heap[u]->delta -= e.w, pop(heap[u]);
            chosen[qi] = e, path[qi++] = u, seen[u] = s;
            res += e.w, u = uf.find(e.a);
            if (seen[u] == s) {
                HeapNode* cyc = 0;
                int end = qi, time = uf.time();
                do cyc = merge(cyc, heap[w = path[--qi]]);
                while (uf.join(u, w));
                u = uf.find(u), heap[u] = cyc, seen[u] = -1;
                cycs.push_front({u, time, {&chosen[qi], &chosen[end]}});
            }
        }
        for (int i = 0; i < qi; ++i) in[uf.find(chosen[i].b)] = chosen[i];
    }

    for (auto& [u, t, comp] : cycs) { // restore sol (optional)
        uf.rollback(t);
        Edge in_edge = in[u];
        for (auto& e : comp) in[uf.find(e.b)] = e;
        in[uf.find(in_edge.b)] = in_edge;
    }
    for (int i = 1; i <= n; ++i) par[i] = in[i].a;
    return {res, par};
}

```

7.8 Math

7.8.1 Number of Spanning Trees

Create an $N \times N$ matrix mat , and for each edge $a \rightarrow b \in G$, do $\text{mat}[a][b]--$, $\text{mat}[b][b]++$ (and $\text{mat}[b][a]--$, $\text{mat}[a][a]++$ if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.8.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

Point.h

Description: class to handle points in the plane. T can be e.g. double or long long. (avoid int.)

21b58d, 27 lines

```

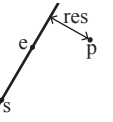
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(Point p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(Point p) const { return tie(x,y)==tie(p.x,p.y); }
    Point operator+(Point p) const { return Point(x+p.x, y+p.y); }
    Point operator-(Point p) const { return Point(x-p.x, y-p.y); }
    Point operator*(T d) const { return Point(x*d, y*d); }
    Point operator/(T d) const { return Point(x/d, y/d); }
    T dot(Point p) const { return x*p.x + y*p.y; }
    T cross(Point p) const { return x*p.y - y*p.x; }
    T cross(Point a, Point b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
    // Angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    Point unit() const { return *this/dist(); } // makes dist()==1
    Point perp() const { return Point(-y, x); } // rotates +90 degrees
    Point normal() const { return perp().unit(); }
    // returns Point rotated 'a' radians ccw around the origin
    Point rotate(double a) const {
        return Point(x*cos(a)-y*sin(a), x*sin(a)+y*cos(a));
    }
    friend ostream& operator<<(ostream& os, Point p) {
        return os << "(" << p.x << ", " << p.y << ")";
    }
};

```

lineDistance.h

Description:

returns the signed distance between Point p and the line containing points a and b. positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. it uses products in intermediate steps so watch out for overflow if using int or long long. using Point3D will always give a non-negative distance. for Point3D, call .dist on the result of the cross product.



"Point.h" 3b34c4, 4 lines

```

template<class P>
double line_dist(const P& a, const P& b, const P& p) {
    return (double) (b-a).cross(p-a) / (b-a).dist();
}

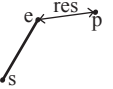
```

SegmentDistance.h

Description:

returns the shortest distance between Point p and the line segment from Point s to e.

Usage: Point<double> a, b(2,2), p(1,1);
bool on_segment = seg_dist(a,b,p) < 1e-10;



"Point.h" 88a0fb, 6 lines

```

typedef Point<double> point_type;
double seg_dist(point_type& s, point_type& e, point_type& p) {
    if (s==e) return (p-s).dist();
    auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s)));
    return ((p-s)*d-(e-s)*t).dist()/d;
}

```

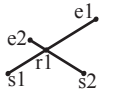
SegmentIntersection.h

Description:

if a unique intersection Point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. if no intersection Point exists an empty vector is returned. if infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. the wrong position will be returned if P is Point<ll> and the intersection Point does not have integer coordinates. products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

Usage: vector<P> inter = seg_inter(s1,e1,s2,e2);

if ((int)(inter).size()==1)
cout << "segments intersect at " << inter[0] << endl;



"Point.h", "OnSegment.h" 120860, 13 lines

```

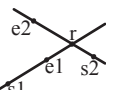
template<class P> vector<P> seg_inter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
        oc = a.cross(b, c), od = a.cross(b, d);
    // checks if intersection is single non-endpoint Point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (on_segment(c, d, a)) s.insert(a);
    if (on_segment(c, d, b)) s.insert(b);
    if (on_segment(a, b, c)) s.insert(c);
    if (on_segment(a, b, d)) s.insert(d);
    return {begin(s), end(s)};
}

```

lineIntersection.h

Description:

if a unique intersection Point of the lines going through s1,e1 and s2,e2 exists {1, Point} is returned. if no intersection Point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. the wrong position will be returned if P is Point<ll> and the intersection Point does not have integer coordinates. products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



Usage: auto res = line_inter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection Point at " << res.second << endl;
"Point.h"b0d826, 8 lines

template<class P>
pair<int, P> line_inter(P s1, P e1, P s2, P e2) {
 auto d = (e1 - s1).cross(e2 - s2);
 if (d == 0) // if parallel
 return {-(s1.cross(e1, s2) == 0), P(0, 0)};
 auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
 return {1, (s1 * p + e1 * q) / d};
}

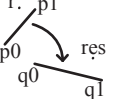
sideOf.h
Description: returns where p is as seen from s towards e . $1/0/-1 \leftrightarrow$ left/on line/right. if the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. it uses products in intermediate steps so watch out for overflow if using int or long long.
Usage: bool left = side_of(p1,p2,q)==1;
"Point.h"9e71fb, 9 lines

template<class P>
int side_of(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int side_of(const P& s, const P& e, const P& p, double eps) {
 auto a = (e-s).cross(p-s);
 double l = (e-s).dist()*eps;
 return (a > l) - (a < -l);
}

OnSegment.h
Description: returns true iff p lies on the line segment from s to e. use (seg_dist(s,e,p)<=epsilon) instead when using Point<double>.
"Point.h"0c1f75, 3 lines

template<class P> bool on_segment(P s, P e, P p) {
 return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}

linearTransformation.h
Description:

apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to Point r.
"Point.h"2aefb4, 6 lines

typedef Point<double> point_type;
point_type linear_transformation(const point_type& p0, const point_type& p1,
 const point_type& q0, const point_type& q1, const point_type& r) {
 point_type dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
 return q0 + point_type((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}

Angle.h
Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). useful for rotational sweeping. sometimes also represents points or vectors.
Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; for (int i = 0; i < (n); ++i) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i
"Point.h"5b6519, 35 lines

struct Angle {
 int x, y;

```
int t;  
Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}  
Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }  
int half() const {  
    assert(x || y);  
    return y < 0 || (y == 0 && x < 0);  
}  
Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }  
Angle t180() const { return {-x, -y, t + half()}; }  
Angle t360() const { return {x, y, t + 1}; }  
};  
bool operator<(Angle a, Angle b) {  
    // add a.dist2() and b.dist2() to also compare distances  
    return make_tuple(a.t, a.half(), a.y * (1l)b.x) <  
        make_tuple(b.t, b.half(), a.x * (1l)b.y);  
}  
  
// given two points, this calculates the smallest Angle between  
// them, i.e., the Angle that covers the defined line segment.  
pair<Angle, Angle> segment_angles(Angle a, Angle b) {  
    if (b < a) swap(a, b);  
    return (b < a.t180() ?  
        make_pair(a, b) : make_pair(b, a.t360()));  
}  
Angle operator+(Angle a, Angle b) { // Point a + vector b  
    Angle r(a.x + b.x, a.y + b.y, a.t);  
    if (a.t180() < r) r.t--;  
    return r.t180() < a ? r.t360() : r;  
}  
Angle angle_diff(Angle a, Angle b) { // Angle b - Angle a  
    int tu = b.t - a.t; a.t = b.t;  
    return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
```

8.2 Circles
CircleIntersection.h
Description: computes the pair of points at which two circles intersect. returns false in case of no intersection.
"Point.h"2f5b4c, 11 lines

typedef Point<double> point_type;
bool circle_inter(point_type a,point_type b,double r1,double r2 ,pair<point_type, point_type>* out) {
 if (a == b) { assert(r1 != r2); return false; }
 point_type vec = b - a;
 double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
 p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
 if (sum*sum < d2 || dif*dif > d2) return false;
 point_type mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
 *out = {mid + per, mid - per};
 return true;
}

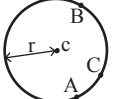
CircleTangents.h
Description: finds the external tangents of two circles, or internal if r2 is negated. can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. to find the tangents of a circle with a Point set r2 to 0.
"Point.h"b0153d, 13 lines

template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
 P d = c2 - c1;
 double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
 if (d2 == 0 || h2 < 0) return {};
 vector<pair<P, P>> out;

```
for (double sign : {-1, 1}) {  
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;  
    out.push_back({c1 + v * r1, c2 + v * r2});  
}  
if (h2 == 0) out.pop_back();  
return out;  
}
```

CirclePolygonIntersection.h
Description: returns the area of the intersection of a circle with a ccw polygon.
Time: $\mathcal{O}(n)$
"../../content/geometry/Point.h"6daa03, 22 lines

typedef Point<double> point_type;
double circle_poly(point_type c, double r, vector<point_type> ps) {
 auto angle_between = [](point_type p, point_type q) {
 return atan2(p.cross(q), p.dot(q));
 };
 auto tri = [&](point_type p, point_type q) {
 auto r2 = r * r / 2;
 point_type d = q - p;
 auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
 auto det = a * a - b;
 if (det <= 0) return angle_between(p, q) * r2;
 auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
 if (t < 0 || 1 <= s) return angle_between(p, q) * r2;
 point_type u = p + d * s, v = q + d * (t-1);
 return angle_between(p, u) * r2 + u.cross(v) / 2
 + angle_between(v, q) * r2;
 };
 auto sum = 0.0;
 for (int i = 0; i < ((int)(ps).size()); ++i)
 sum += tri(ps[i] - c, ps[(i + 1) % (int)(ps).size()] - c);
 return sum;
}

circumcircle.h
Description:

the circumcircle of a triangle is the circle intersecting all three vertices. cc.radius returns the radius of the circle going through points A, B and C and cc.center returns the center of the same circle.
"Point.h"89349f, 9 lines

typedef Point<double> circle_point;
double cc_radius(const circle_point& A, const circle_point& B, const circle_point& C) {
 return (B-A).dist()*(C-B).dist()*(A-C).dist() /
 abs((B-A).cross(C-A)) / 2;
}
circle_point cc_center(const circle_point& A, const circle_point& B, const circle_point& C) {
 circle_point b = C-A, c = B-A;
 return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}

MinimumEnclosingCircle.h
Description: computes the minimum circle that encloses a set of points.
Time: expected $\mathcal{O}(n)$
"circumcircle.h"ac40aa, 18 lines

pair<circle_point, double> minimum_enclosing_circle(vector<circle_point> ps) {
 shuffle(begin(ps), end(ps), mt19937(time(0)));
 circle_point o = ps[0];
 double r = 0;
 const double EPS = 1 + 1e-8;

```
for (int i = 0; i < ((int) (ps).size()); ++i) if ((o - ps[i]).
    dist() > r * EPS) {
    o = ps[i], r = 0;
    for (int j = 0; j < (i); ++j) if ((o - ps[j]).dist() > r *
        EPS) {
        o = (ps[i] + ps[j]) / 2;
        r = (o - ps[i]).dist();
        for (int k = 0; k < (j); ++k) if ((o - ps[k]).dist() > r
            * EPS) {
            o = cc_center(ps[i], ps[j], ps[k]);
            r = (o - ps[i]).dist();
        }
    }
}
return {o, r};
}
```

8.3 Polygons

InsidePolygon.h

Description: returns true if p lies within the polygon. if strict is true, it returns false for points on the boundary. the algorithm uses products in intermediate steps so watch out for overflow.

Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};

bool in = in.polygon(v, P{3, 3}, false);

Time: $\mathcal{O}(n)$

```
"Point.h", "OnSegment.h", "SegmentDistance.h" 5ee6ed, 11 lines
template<class P>
bool in_polygon(vector<P> &p, P a, bool strict = true) {
    int cnt = 0, n = (int) (p).size();
    for (int i = 0; i < (n); ++i) {
        P q = p[(i + 1) % n];
        if (on_segment(p[i], q, a)) return !strict;
        //or: if (seg_dist(p[i], q, a) <= eps) return !strict;
        cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
    }
    return cnt;
}
```

PolygonArea.h

Description: returns twice the signed area of a polygon. clockwise enumeration gives negative area. watch out for overflow if using int as T!

```
"Point.h" 6b6f75, 6 lines
template<class T>
T polygon_area2(vector<Point<T>>& v) {
    T a = v.back().cross(v[0]);
    for (int i = 0; i < ((int) (v).size()-1); ++i) a += v[i].cross
        (v[i+1]);
    return a;
}
```

PolygonCenter.h

Description: returns the center of mass for a polygon.

Time: $\mathcal{O}(n)$

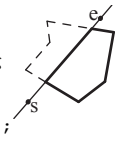
```
"Point.h" c96ede, 9 lines
typedef Point<double> point_type;
point_type polygon_center(const vector<point_type>& v) {
    point_type res(0, 0); double area2 = 0;
    for (int i = 0, j = (int) (v).size() - 1; i < (int) (v).size();
        j = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        area2 += v[j].cross(v[i]);
    }
    return res / area2 / 3;
}
```

PolygonCut.h

Description: returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.

Usage: vector<point_type> p = ...;

p = polygon_cut(p, point_type(0,0), point_type(1,0));



```
"Point.h" 832f26, 13 lines
typedef Point<double> point_type;
vector<point_type> polygon_cut(const vector<point_type>& poly,
    point_type s, point_type e) {
    vector<point_type> res;
    for (int i = 0; i < ((int) (poly).size()); ++i) {
        point_type cur = poly[i], prev = i ? poly[i-1] : poly.back
            ();
        auto a = s.cross(e, cur), b = s.cross(e, prev);
        if ((a < 0) != (b < 0))
            res.push_back(cur + (prev - cur) * (a / (a - b)));
        if (a < 0)
            res.push_back(cur);
    }
    return res;
}
```

ConvexHull.h

Description: returns a vector of the points of the convex hull in counter-clockwise order. points on the edge of the hull between two other points are not considered part of the hull.

Time: $\mathcal{O}(n \log n)$

```
"Point.h" 63a06f, 13 lines
template<class T>
vector<Point<T>> convex_hull(vector<Point<T>> pts) {
    if ((int) (pts).size() <= 1) return pts;
    sort(begin(pts), end(pts));
    vector<Point<T>> h((int) (pts).size()+1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(begin(pts), end(pts))
        )
        for (Point<T> p : pts) {
            while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
            h[t++] = p;
        }
    return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

HullDiameter.h

Description: returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

Time: $\mathcal{O}(n)$

```
"Point.h" 66028a, 12 lines
typedef Point<ll> point_type;
array<point_type, 2> hull_diameter(vector<point_type> points) {
    int n = (int) points.size(), j = n < 2 ? 0 : 1;
    pair<ll, array<point_type, 2>> res({0, {points[0], points[0]}});
    for (int i = 0; i < (j); ++i)
        for (; j = (j + 1) % n) {
            res = max(res, {(points[i] - points[j]).dist2(), {points[
                i], points[j]}});
            if ((points[(j + 1) % n] - points[j]).cross(points[i + 1]
                - points[i]) >= 0)
                break;
        }
    return res.second;
}
```

PointInsideHull.h

Description: determine whether a Point t lies inside a convex hull (CCW order, with no collinear points). returns true if Point lies within the hull. if strict is true, points on the boundary aren't included.

Time: $\mathcal{O}(\log N)$

"Point.h", "SideOf.h", "OnSegment.h" 62f38d, 14 lines

typedef Point<ll> point_type;

```
bool in_hull(const vector<point_type>& l, point_type p, bool
    strict = true) {
    int a = 1, b = (int) (l).size() - 1, r = !strict;
    if ((int) (l).size() < 3) return r && on_segment(l[0], l.back
        (), p);
    if (side_of(l[0], l[a], l[b]) > 0) swap(a, b);
    if (side_of(l[0], l[a], p) >= r || side_of(l[0], l[b], p) <= -
        r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (side_of(l[0], l[c], p) > 0 ? b : a) = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

Description: line-convex polygon intersection. the polygon must be ccw and have no collinear points. line_hull(line, poly) returns a pair describing the intersection of a line with the polygon: • (0,0) if no collision, • (i,0) if touching the corner i, • (i,i) if along side (i,i+1), • (i,j) if crossing sides (i,i+1) and (j,j+1). in the last case, if a corner i is crossed, this is treated as happening on side (i,i+1). the points are returned in the same order as the line hits the polygon. Polygon vertex and side indices are 1-indexed. extreme_vertex returns the index of a hull vertex with maximum projection onto a line.

Time: $\mathcal{O}(\log n)$

```
"Point.h" b4571c, 48 lines
template <class P> int extreme_vertex_index(vector<P>& poly, P
    dir) {
    int n = (int) (poly).size(), lo = 0, hi = n;
    auto compare = [&](int i, int j) {
        return sgn(dir.perp().cross(poly[i % n] - poly[j % n]));
    };
    auto is_extreme = [&](int i) {
        return compare(i + 1, i) >= 0
            && compare(i, i - 1 + n) < 0;
    };
    if (is_extreme(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (is_extreme(m)) return m;
        int ls = compare(lo + 1, lo), ms = compare(m + 1, m);
        (ls < ms || (ls == ms && ls == compare(lo, m)) ? hi : lo) =
            m;
    }
    return lo;
}

template <class P> int extreme_vertex(vector<P>& poly, P dir) {
    return extreme_vertex_index(poly, dir) + 1;
}

template <class P>
array<int, 2> line_hull(P a, P b, vector<P>& poly) {
    auto compare_line = [&](int i) { return sgn(a.cross(poly[i],
        b)); };
    int end_a = extreme_vertex_index(poly, (a - b).perp());
    int end_b = extreme_vertex_index(poly, (b - a).perp());
    if (compare_line(end_a) < 0 || compare_line(end_b) > 0)
```

```

    return {0, 0};
array<int, 2> res;
for (int i = 0; i < (2); ++i) {
    int lo = end_b, hi = end_a, n = (int)(poly).size();
    while ((lo + 1) % n != hi) {
        int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
        (compare_line(m) == compare_line(end_b) ? lo : hi) = m;
    }
    res[i] = (lo + !compare_line(hi)) % n;
    swap(end_a, end_b);
}
if (res[0] == res[1]) return {res[0] + 1, 0};
if (!compare_line(res[0]) && !compare_line(res[1]))
    switch ((res[0] - res[1] + (int)(poly).size() + 1) % (int)(poly).size()) {
        case 0: return {res[0] + 1, res[0] + 1};
        case 2: return {res[1] + 1, res[1] + 1};
    }
return {res[0] + 1, res[1] + 1};
}

```

8.4 Misc. Point Set Problems

ClosestPair.h

Description: finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$

"Point.h"	de574b, 17 lines
<pre> typedef Point<ll> point_type; pair<point_type, point_type> closest(vector<point_type> v) { assert((int)(v).size() > 1); set<point_type> candidates; sort(begin(v), end(v), [](point_type a, point_type b) { return a.y < b.y; }); pair<ll, pair<point_type, point_type>> ret{LLONG_MAX, { point_type(), point_type()}}; int j = 0; for (point_type p : v) { point_type d[1 + (ll)sqrt(ret.first), 0]; while (v[j].y <= p.y - d.x) candidates.erase(v[j++]); auto lo = candidates.lower_bound(p - d), hi = candidates. upper_bound(p + d); for (; lo != hi; ++lo) ret = min(ret, {(lo - p).dist2(), {lo, p}}); candidates.insert(p); } return ret.second; } </pre>	

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

"Point.h"	39fb26, 63 lines
<pre> typedef long long T; typedef Point<T> point_type; const T INF = numeric_limits<T>::max(); bool on_x(const point_type& a, const point_type& b) { return a. x < b.x; } bool on_y(const point_type& a, const point_type& b) { return a. y < b.y; } struct KdNode { point_type pt; // if this is a leaf, the single Point in it T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds KdNode *first = 0, *second = 0; T distance(const point_type& p) { // min squared distance to a Point T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x); T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y); </pre>	

```

    return (point_type(x,y) - p).dist2();
}

KdNode(vector<point_type>&& vp) : pt(vp[0]) {
    for (point_type p : vp) {
        x0 = min(x0, p.x); x1 = max(x1, p.x);
        y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
        // split on x if width >= height (not ideal...)
        sort(begin(vp), end(vp), x1 - x0 >= y1 - y0 ? on_x : on_y
            );
        // divide by taking half the array for each child (not
        // best performance with many duplicates in the middle)
        int half = (int)(vp).size()/2;
        first = new KdNode({vp.begin(), vp.begin() + half});
        second = new KdNode({vp.begin() + half, vp.end()});
    }
}

struct KdTree {
    KdNode* root;
    KdTree(const vector<point_type>& vp) : root(new KdNode({begin
        (vp), end(vp)})) {}

    pair<T, point_type> search(KdNode *current, const point_type&
        p) {
        if (!current->first) {
            // uncomment if we should not find the Point itself:
            // if (p == node->pt) return {INF, point_type()};
            return make_pair((p - current->pt).dist2(), current->pt);
        }

        KdNode *f = current->first, *s = current->second;
        T bfirst = f->distance(p), bsec = s->distance(p);
        if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

        // search closest side first, other side if needed
        auto best = search(f, p);
        if (bsec < best.first)
            best = min(best, search(s, p));
        return best;
    }

    // find nearest Point to a Point, and its squared distance
    // (requires an arbitrary operator< for Point)
    pair<T, point_type> nearest(const point_type& p) {
        return search(root, p);
    }
};

```

FastDelaunay.h

Description: fast delaunay triangulation. each circumcircle contains none of the input points. there must be no duplicate points. if all points are on a line, no triangles will be returned. should work for doubles as well, though there may be precision issues in 'circ'. returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

"Point.h"	09bbc6, 110 lines
<pre> typedef Point<ll> delaunay_point; typedef struct QuadEdge* quad_edge; typedef __int128_t wide_int; // (can be ll if coords are < 2e4) const delaunay_point ARBITRARY_POINT(LLONG_MAX, LLONG_MAX); struct QuadEdge { quad_edge rot, o; delaunay_point p = ARBITRARY_POINT; bool mark = false; delaunay_point& destination() { return r()->p; } </pre>	

```

    quad_edge& r() { return rot->rot; }
    quad_edge prev() { return rot->o->rot; }
    quad_edge next() { return r()->prev(); }
} *free_edges;

```

```

bool circ(delaunay_point p, delaunay_point a, delaunay_point b,
    delaunay_point c) {
    wide_int p2 = p.dist2(), da = a.dist2() - p2,
    db = b.dist2() - p2, dc = c.dist2() - p2;
    return p.cross(a, b) * dc + p.cross(b, c) * da
        + p.cross(c, a) * db > 0;
}

quad_edge make_edge(delaunay_point orig, delaunay_point dest) {
    quad_edge r = free_edges ? free_edges : new QuadEdge{new
        QuadEdge{new QuadEdge{new QuadEdge{0}}}};
    free_edges = r->o; r->r()->r() = r;
    for (int i = 0; i < (4); ++i) r = r->rot, r->p =
        ARBITRARY_POINT, r->o = i & 1 ? r : r->r();
    r->p = orig; r->destination() = dest;
    return r;
}

void splice(quad_edge a, quad_edge b) {
    swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}

quad_edge connect(quad_edge a, quad_edge b) {
    quad_edge q = make_edge(a->destination(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}

pair<quad_edge, quad_edge> triangulate_rec(const vector<
    delaunay_point>& s) {
    if ((int)(s).size() <= 3) {
        quad_edge a = make_edge(s[0], s[1]), b = make_edge(s[1], s.
            back());
        if ((int)(s).size() == 2) return { a, a->r() };
        splice(a->r(), b);
        auto side = s[0].cross(s[1], s[2]);
        quad_edge c = side ? connect(b, a) : 0;
        return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
    }
}

```

```

quad_edge left, right, left_outer, right_outer;
int half = (int)(s).size() / 2;
tie(left_outer, left) = triangulate_rec({begin(s), end(s) -
    half});
tie(right, right_outer) = triangulate_rec(
    {(int)s.size() - half + begin(s), end(s)});
while ((right->p.cross(left->destination(), left->p) < 0
    && (left == left->next()))
    || (left->p.cross(right->destination(), right->p) > 0
    && (right == right->r()->o)));
quad_edge base = connect(right->r(), left);
if (left->p == left_outer->p) left_outer = base->r();
if (right->p == right_outer->p) right_outer = base;
auto valid = [&](quad_edge edge) {
    return edge->destination().cross(base->destination(), base
        ->p) > 0;
};

auto discard_edges = [&](quad_edge initial, auto next_edge) {
    quad_edge edge = next_edge(initial);
    if (valid(edge)) while (circ(next_edge(edge)->destination()
        ,
        base->destination(), base->p, edge->destination())) {
        quad_edge next = next_edge(edge);
        splice(edge, edge->prev());
        splice(edge->r(), edge->r()->prev());
        edge->o = free_edges; free_edges = edge; edge = next;
    }
}

```

```
    }
    return edge;
};
for (;;) {
    quad_edge left_candidate = discard_edges(base->r()),
    [] (quad_edge edge) { return edge->o; });
    quad_edge right_candidate = discard_edges(base,
    [] (quad_edge edge) { return edge->prev(); });
    if (!valid(left_candidate) && !valid(right_candidate))
        break;
    if (!valid(left_candidate) || (valid(right_candidate)
        && circ(right_candidate->destination(), right_candidate
        ->p,
        left_candidate->destination(), left_candidate->p)))
        base = connect(right_candidate, base->r());
    else
        base = connect(base->r(), left_candidate->r());
}
return {left_outer, right_outer};
}
```

```
vector<delaunay_point> triangulate(vector<delaunay_point> pts)
{
    sort(begin(pts), end(pts));  assert(unique(begin(pts), end(
        pts)) == pts.end());
    if ((int)(pts).size() < 2) return {};
    quad_edge e = triangulate_rec(pts).first;
    vector<quad_edge> q = {e};
    int qi = 0;
    while (e->o->destination().cross(e->destination(), e->p) < 0)
        e = e->o;
    auto add_face = [&](quad_edge edge) {
        quad_edge current = edge;
        do {
            current->mark = true;
            pts.push_back(current->p);
            q.push_back(current->r());
            current = current->next();
        } while (current != edge);
    };
    add_face(e); pts.clear();
    while (qi < (int)q.size())
        if (!(e = q[qi++])->mark) add_face(e);
    return pts;
}
```

8.5 3D

PolyhedronVolume.h

Description: magic formula for the volume of a polyhedron. faces should Point outwards.

```
template<class V, class L>
double signed_poly_volume(const V& p, const L& trilst) {
    double v = 0;
    for (auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
    return v / 6;
}
```

Point3D.h

Description: class to handle points in 3D space. T can be e.g. double or long long.

```
template<class T> struct Point3D {
    typedef const Point3D& point_reference;
    T x, y, z;
    explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {}
    bool operator<(point_reference p) const {
        return tie(x, y, z) < tie(p.x, p.y, p.z); }
    bool operator==(point_reference p) const {
```

```
        return tie(x, y, z) == tie(p.x, p.y, p.z); }
    Point3D operator+(point_reference p) const { return Point3D(x
        +p.x, y+p.y, z+p.z); }
    Point3D operator-(point_reference p) const { return Point3D(x
        -p.x, y-p.y, z-p.z); }
    Point3D operator*(T d) const { return Point3D(x*d, y*d, z*d);
    }
    Point3D operator/(T d) const { return Point3D(x/d, y/d, z/d);
    }
    T dot(point_reference p) const { return x*p.x + y*p.y + z*p.z
    ; }
    Point3D cross(point_reference p) const {
        return Point3D(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x)
        ;
    }
    T dist2() const { return x*x + y*y + z*z; }
    double dist() const { return sqrt((double)dist2()); }
    //azimuthal Angle (longitude) to x-axis in interval [-pi, pi]
    double phi() const { return atan2(y, x); }
    //zenith Angle (latitude) to the z-axis in interval [0, pi]
    double theta() const { return atan2(sqrt(x*x+y*y),z); }
    Point3D unit() const { return *this/(T)dist(); } //makes dist
    ()=1
    //returns unit vector normal to *this and p
    Point3D normal(Point3D p) const { return cross(p).unit(); }
    //returns Point rotated 'Angle' radians ccw around axis
    Point3D rotate(double angle, Point3D axis) const {
        double s = sin(angle), c = cos(angle); Point3D u = axis.
            unit();
        return u.dot(u)*(1-c) + (*this)*c - cross(u)*s;
    }
};
```

3dHull.h

Description: computes all faces of the 3-dimension hull of a Point set. *no four points must be coplanar*, or else random results will be returned. all faces will Point outwards. Face vertex indices are 1-indexed.

```
Time:  $\mathcal{O}(n^2)$ 
"Point3D.h" a3c9e3, 58 lines

typedef Point3D<double> hull_point_3d;

struct EdgeFaces {
    void ins(int x) { (a == -1 ? a : b) = x; }
    void rem(int x) { (a == x ? a : b) = -1; }
    int cnt() { return (a != -1) + (b != -1); }
    int a, b;
};
```

struct Face { hull_point_3d normal; int a, b, c; };

```
vector<Face> hull_3d(const vector<hull_point_3d>& points) {
    assert((int)points.size() >= 4);
    vector<vector<EdgeFaces>> edges((int)points.size(),
        vector<EdgeFaces>((int)points.size(), {-1, -1}));
    vector<Face> faces;
    auto mf = [&](int i, int j, int k, int l) {
        hull_point_3d q = (points[j] - points[i]).cross(points[k] -
            points[i]);
        if (q.dot(points[l]) > q.dot(points[i]))
            q = q * -1;
        Face current{q, i, j, k};
        edges[current.a][current.b].ins(k);
        edges[current.a][current.c].ins(j);
        edges[current.b][current.c].ins(i);
        faces.push_back(current);
    };
    for (int i = 0; i < (4); ++i) for (int j = i+1; j < (4); ++j)
        for (int k = j+1; k < (4); ++k)
            mf(i, j, k, 6 - i - j - k);
```

```
for (int i = 4; i < (int)points.size(); ++i) {
    for (int j = 0; j < (int)faces.size(); ++j) {
        Face current = faces[j];
        if (current.normal.dot(points[i])
            > current.normal.dot(points[current.a])) {
            edges[current.a][current.b].rem(current.c);
            edges[current.a][current.c].rem(current.b);
            edges[current.b][current.c].rem(current.a);
            swap(faces[j--], faces.back());
            faces.pop_back();
        }
    }
    int nw = (int)faces.size();
    for (int j = 0; j < (nw); ++j) {
        Face current = faces[j];
        if (edges[current.a][current.b].cnt() != 2)
            mf(current.a, current.b, i, current.c);
        if (edges[current.a][current.c].cnt() != 2)
            mf(current.a, current.c, i, current.b);
        if (edges[current.b][current.c].cnt() != 2)
            mf(current.b, current.c, i, current.a);
    }
}
for (Face& current : faces) if ((points[current.b] - points[
    current.a]).cross(
    points[current.c] - points[current.a]).dot(current.normal)
    <= 0)
    swap(current.c, current.b);
for (Face& face : faces) ++face.a, ++face.b, ++face.c;
return faces;
};
```

sphericalDistance.h

Description: returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). all angles measured in radians. the algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

```
double spherical_distance(double f1, double t1,
    double f2, double t2, double radius) {
    double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
    double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
    double dz = cos(t2) - cos(t1);
    double d = sqrt(dx*dx + dy*dy + dz*dz);
    return radius*2*asin(d/2);
}
```

Strings (9)

KMP.h

Description: pi(s)[x] computes the length of the longest proper prefix of s ending at 1-indexed position x. The returned vector has a dummy at index 0. can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$

```
vector<int> pi(const string& s) {
    vector<int> p(s.size() + 1);
    for (int i = 2; i <= (int)s.size(); ++i) {
        int g = p[i-1];
        while (g && s[i-1] != s[g]) g = p[g];
        p[i] = g + (s[i-1] == s[g]);
    }
    return p;
}
```

```

}

vector<int> match(const string& s, const string& pat) {
    vector<int> p = pi(pat + '\0' + s), res;
    for (int i = (int)p.size()-(int)s.size(); i < (int)p.size(); ++i)
        if (p[i] == (int)(pat).size()) res.push_back(i - 2 * (int)(pat).size());
    return res;
}

```

Zfunc.h

Description: z[i] computes the longest common prefix of the suffix starting at 1-indexed position i and the full string, except z[1] = 0. The returned vector has index 0 unused. (abacaba -> 0010301)
Time: $\mathcal{O}(n)$

```

vector<int> z_function(const string& s) {
    vector<int> z((int)s.size());
    int l = -1, r = -1;
    for (int i = 1; i < (int)s.size(); ++i) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while (i + z[i] < (int)s.size() && s[i + z[i]] == s[z[i]])
            z[i]++;
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
    z.insert(begin(z), 0);
    return z;
}

```

Manacher.h

Description: for each 1-indexed position in a string, computes p[0][i] = half the longest even palindrome around i, and p[1][i] = the longest odd palindrome's half length rounded down. Both vectors have index 0 unused.
Time: $\mathcal{O}(N)$

```

array<vector<int>, 2> manacher(const string& s) {
    int n = (int)(s).size();
    array<vector<int>,2> p = {vector<int>(n+1), vector<int>(n)};
    for (int z = 0; z < (2); ++z) for (int i=0,l=0,r=0; i < n; i++) {
        int t = r-i!+z;
        if (i<r) p[z][i] = min(t, p[z][l+t]);
        int left = i-p[z][i], right = i+p[z][i]-!z;
        while (left>=l && right+1<n && s[left-1] == s[right+1])
            p[z][i]++, left--, right++;
        if (right>r) l=left, r=right;
    }
    array<vector<int>, 2> result = {vector<int>(n+1), vector<int>(n+1)};
    for (int z = 0; z < 2; ++z) for (int i = 0; i < n; ++i)
        result[z][i+1] = p[z][i];
    return result;
}

```

MinRotation.h

Description: finds the lexicographically smallest rotation of a string.
Usage: rotate(v.begin(), v.begin()+min.rotation(v)-1, v.end());
Time: $\mathcal{O}(N)$

```

int min_rotation(string s) {
    int a=0, n=(int)(s).size(); s += s;
    for (int b = 0; b < n; ++b) for (int k = 0; k < n; ++k) {
        if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
        if (s[a+k] > s[b+k]) {a = b; break;}
    }
    return a + 1;
}

```

SuffixArray.h

Description: builds suffix array for a string. sa[i] is the 1-indexed starting position of the suffix which is i'th in the sorted suffix array. the returned vectors have size n+2 with index 0 unused, and sa[1] = n+1 is the empty suffix. the lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. the input string must not contain any nul chars.
Time: $\mathcal{O}(n \log n)$

```

392716, 24 lines
struct SuffixArray {
    vector<int> sa, lcp;
    SuffixArray(string s, int lim=256) { // or vector<int>
        s.push_back(0); int n = (int)(s).size(), k = 0, a, b;
        vector<int> x(begin(s), end(s)), y(n), ws(max(n, lim));
        sa = lcp = y, iota(begin(sa), end(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
            p = j, iota(begin(y), end(y), n - j);
            for (int i = 0; i < (n); ++i) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(begin(ws), end(ws), 0);
            for (int i = 0; i < (n); ++i) ws[x[i]]++;
            for (int i = 1; i < (lim); ++i) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            for (int i = 1; i < (n); ++i) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
        }
        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
            for (k && k--, j = sa[x[i] - 1];
                s[i + k] == s[j + k]; k++);
        sa.insert(begin(lcp), 0);
        for (int i = 1; i < (int)sa.size(); ++i) ++sa[i];
    }
};

```

SuffixTree.h

Description: ukkonen's algorithm for online suffix tree construction. Public node IDs and string positions are 1-indexed. edge_range(node) returns an inclusive-inclusive substring range; child returns 0 if absent. The root has ID 1 and no edge range. to get a complete tree, append a dummy symbol - otherwise it may contain an incomplete path (still useful for substring matching, though).
Time: $\mathcal{O}(28N)$

```

6df69f, 88 lines
struct SuffixTree {
private:
    enum { MAX_N = 200010, ALPHABET_SIZE = 28 }; // MAXN ~ 2*
        maxlen+10
    string text;
    int next_node[MAX_N][ALPHABET_SIZE], left[MAX_N], right[MAX_N];
    int parent[MAX_N], suffix_link[MAX_N];
    int active_node = 0, active_position = 0, node_total = 2;
    pii best;

    int to_index(char c) { return c - 'a'; }
    void add(int position, int character) { suffix:
        if (right[active_node] <= active_position) {
            if (next_node[active_node][character] == -1) {
                next_node[active_node][character] = node_total;
                left[node_total] = position; parent[node_total++] =
                    active_node;
                active_node = suffix_link[active_node];
                active_position = right[active_node]; goto suffix;
            }
            active_node = next_node[active_node][character];
            active_position = left[active_node];
        }
    }
};

```

```

if (active_position == -1 || character == to_index(text[
    active_position]))
    active_position++;
else {
    left[node_total+1] = position; parent[node_total+1] =
        node_total;
    left[node_total] = left[active_node]; right[node_total] =
        active_position;
    parent[node_total] = parent[active_node];
    next_node[node_total][character] = node_total + 1;
    next_node[node_total][to_index(text[active_position])] =
        active_node;
    left[active_node] = active_position; parent[active_node]
        = node_total;
    next_node[parent[node_total]][to_index(text[left[
        node_total]])] = node_total;
    active_node = suffix_link[parent[node_total]];
    active_position = left[node_total];
    while (active_position < right[node_total]) {
        active_node = next_node[active_node][to_index(text[
            active_position])];
        active_position += right[active_node] - left[
            active_node];
    }
    if (active_position == right[node_total]) suffix_link[
        node_total] = active_node;
    else suffix_link[node_total] = node_total + 2;
    active_position = right[active_node] - (active_position -
        right[node_total]);
    node_total += 2; goto suffix;
}
}
int lcs_dfs(int node, int first_separator, int
    second_separator, int old_length) {
    if (left[node] <= first_separator && first_separator <
        right[node]) return 1;
    if (left[node] <= second_separator && second_separator <
        right[node]) return 2;
    int mask = 0;
    int length = node ? old_length + right[node] - left[node] :
        0;
    for (int c = 0; c < ALPHABET_SIZE; ++c) if (next_node[node
        ][c] != -1)
        mask |= lcs_dfs(next_node[node][c], first_separator,
            second_separator, length);
    if (mask == 3)
        best = max(best, {length, right[node] - length + 1});
    return mask;
}

```

```

public:
SuffixTree(string text) : text(text) {
    fill(right, right+MAX_N, (int)text.size());
    memset(suffix_link, 0, sizeof suffix_link);
    memset(next_node, -1, sizeof next_node);
    fill(next_node[1], next_node[1]+ALPHABET_SIZE, 0);
    suffix_link[0] = 1;
    left[0] = left[1] = -1;
    right[0] = right[1] = parent[0] = parent[1] = 0;
    for (int i = 0; i < (int)text.size(); ++i) add(i, to_index(
        text[i]));
}
int size() const { return node_total - 1; }
int child(int node, char c) const {
    int internal_node = node == 1 ? 0 : node;
    int result = next_node[internal_node][c - 'a'];
    return result == -1 ? 0 : result;
}
int parent_of(int node) const {

```

```
int result = parent[node];
return result == 0 ? 1 : result;
}
pii edge_range(int node) const {
    assert(node > 1);
    return {left[node] + 1, right[node]};
}

// find longest common substring; returns (length, 1-indexed
// start in s)
static pii lcs(string s, string t) {
    auto tree = make_unique<SuffixTree>(s + (char)('z' + 1) + t
    + (char)('z' + 2));
    tree->lcs_dfs(0, (int)s.size(), (int)s.size() + 1 + (int)t.
    size(), 0);
    return tree->best;
}
};
```

Hashing.h

Description: self-explanatory methods for string hashing. 2d6838, 48 lines

```
// arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. thue-morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull HashValue;" instead if you think test data is
// random,
// or work mod 10^9+7 if the birthday paradox is not a problem.
typedef uint64_t ull;
struct HashValue {
    ull x; HashValue(ull x=0) : x(x) {}
    HashValue operator+(HashValue o) { return x + o.x + (x + o.x
    < x); }
    HashValue operator-(HashValue o) { return *this + ~o.x; }
    HashValue operator*(HashValue o) { auto m = (__uint128_t)x *
    o.x;
    return HashValue((ull)m) + (ull)(m >> 64); }
    ull get() const { return x + !~x; }
    bool operator==(HashValue o) const { return get() == o.get(); }
    bool operator<(HashValue o) const { return get() < o.get(); }
};
static const HashValue BASE = (1ll)1e11+3;
```

```
struct RollingHash {
    vector<HashValue> hashes, powers;
    RollingHash(const string& str) : hashes((int)str.size()+1),
    powers(hashes) {
        powers[0] = 1;
        for (int i = 0; i < ((int)(str).size()); ++i)
            hashes[i+1] = hashes[i] * BASE + str[i],
            powers[i+1] = powers[i] * BASE;
    }
    HashValue query(int left, int right) { // 1-indexed [left,
    right]
        return hashes[right] - hashes[left - 1] * powers[right -
        left + 1];
    }
};
```

```
vector<HashValue> get_hashes(const string& str, int length) {
    if ((int)str.size() < length) return vector<HashValue>(1);
    HashValue hash = 0, power = 1;
    for (int i = 0; i < (length); ++i)
        hash = hash * BASE + str[i], power = power * BASE;
    vector<HashValue> result = {HashValue{}, hash};
    for (int i = length; i < ((int)(str).size()); ++i) {
        result.push_back(hash = hash * BASE + str[i] - power * str[
        i-length]);
    }
```

```
}
return result;
}

HashValue hash_string(const string& s) {
    HashValue hash{};
    for (char c : s) hash = hash * BASE + c;
    return hash;
}
```

AhoCorasick.h

Description: aho-corasick automaton, used for multiple pattern matching. initialize with AhoCorasick ac(patterns). Pattern and text positions are numbered from 1. find(word) returns a vector with index 0 unused; each entry is the index of the longest pattern ending there, or 0 if none. find_all(−, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). duplicate patterns are allowed; empty patterns are not. to find the longest words that start at each position, reverse all input. for large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.

Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. find_all is $\mathcal{O}(NM)$. 033632, 68 lines

```
struct AhoCorasick {
private:
    enum { ALPHABET_SIZE = 26, FIRST_CHARACTER = 'A' }; // change
    this!
    struct Node {
        // (nmatches is optional)
        int back, next[ALPHABET_SIZE], start = 0, end = 0, nmatches
        = 0;
        Node(int v) { memset(next, v, sizeof(next)); }
    };
    vector<Node> nodes;
    vector<int> backp = vector<int>(1);
    void insert(const string& s, int j) {
        assert(!s.empty());
        int n = 0;
        for (char c : s) {
            int& m = nodes[n].next[c - FIRST_CHARACTER];
            if (m == -1) { n = m = (int)nodes.size(); nodes.
            emplace_back(-1); }
            else n = m;
        }
        if (nodes[n].end == 0) nodes[n].start = j;
        backp.push_back(nodes[n].end);
        nodes[n].end = j;
        nodes[n].nmatches++;
    }
public:
    AhoCorasick(const vector<string>& pat) : nodes(1, -1) {
        for (int i = 0; i < (int)pat.size(); ++i) insert(pat[i], i
        + 1);
        nodes[0].back = (int)nodes.size();
        nodes.emplace_back(0);

        queue<int> q;
        for (q.push(0); !q.empty(); q.pop()) {
            int n = q.front(), prev = nodes[n].back;
            for (int i = 0; i < ALPHABET_SIZE; ++i) {
                int &ed = nodes[n].next[i], y = nodes[prev].next[i];
                if (ed == -1) ed = y;
                else {
                    nodes[ed].back = y;
                    (nodes[ed].end == 0 ? nodes[ed].end : backp[nodes[ed]
                    ].start)
                    = nodes[y].end;
                    nodes[ed].nmatches += nodes[y].nmatches;
                    q.push(ed);
                }
            }
        }
    }
};
```

```
}
}
}
}
vector<int> find(string word) {
    int n = 0;
    vector<int> res(1); // 1l count = 0;
    for (char c : word) {
        n = nodes[n].next[c - FIRST_CHARACTER];
        res.push_back(nodes[n].end);
        // count += N[n].nmatches;
    }
    return res;
}
vector<vector<int>> find_all(vector<string>& pat, string word
) {
    vector<int> r = find(word);
    vector<vector<int>> res(word.size() + 1);
    for (int i = 1; i <= (int)word.size(); ++i) {
        int ind = r[i];
        while (ind) {
            res[i - (int)pat[ind-1].size() + 1].push_back(ind);
            ind = backp[ind];
        }
    }
    return res;
}
};
```

Various (10)

10.1 Intervals

IntervalContainer.h

Description: add and remove intervals from a set of disjoint intervals. will merge the added interval with any overlapping intervals in the set when adding. intervals are [inclusive, inclusive].

Time: $\mathcal{O}(\log N)$ 6b24c6, 23 lines

```
set<pii>::iterator add_interval(set<pii>& intervals, int left,
int right) {
    if (left > right) return intervals.end();
    auto it = intervals.lower_bound({left, right}), before = it;
    while (it != intervals.end() && it->first <= right + 1) {
        right = max(right, it->second);
        before = it = intervals.erase(it);
    }
    if (it != intervals.begin() && (--it)->second + 1 >= left) {
        left = min(left, it->first);
        right = max(right, it->second);
        intervals.erase(it);
    }
    return intervals.insert(before, {left, right});
}
```

```
void remove_interval(set<pii>& intervals, int left, int right)
{
    if (left > right) return;
    auto it = add_interval(intervals, left, right);
    int old_left = it->first, old_right = it->second;
    intervals.erase(it);
    if (old_left < left) intervals.emplace(old_left, left - 1);
    if (right < old_right) intervals.emplace(right + 1, old_right
    );
}
```

IntervalCover.h

Description: compute indices of smallest set of intervals covering another interval. intervals and the goal are [inclusive, inclusive] integer ranges. returns 1-indexed interval positions, or an empty set on failure (or if the goal is empty).

Time: $\mathcal{O}(N \log N)$

```
vector<int> cover(pii goal, vector<pii> intervals) {
    vector<int> order((int)intervals.size(), result;
    iota(begin(order), end(order), 0);
    sort(begin(order), end(order), [&](int a, int b) {
        return intervals[a] < intervals[b];
    });
    int current = goal.first;
    int at = 0;
    while (current <= goal.second) {
        pii best = {current - 1, -1};
        while (at < (int)intervals.size()
            && intervals[order[at]].first <= current) {
            best = max(best, {intervals[order[at]].second, order[at]});
            at++;
        }
        if (best.second == -1) return {};
        current = best.first + 1;
        result.push_back(best.second + 1);
    }
    return result;
}
```

ConstantIntervals.h

Description: split a monotone function on [from, to] into a minimal set of inclusive intervals on which it has the same value. runs a callback g for each such interval.

Usage: constant.intervals(1, n, f, [&](int lo, int hi, T val){...});

Time: $\mathcal{O}(k \log \frac{n}{k})$

```
template<class F, class G, class T>
void constant_intervals_rec(int from, int to, F& f, G& g,
    int& start, T& previous, T current) {
    if (previous == current) return;
    if (from == to) {
        g(start, to - 1, previous);
        start = to; previous = current;
    } else {
        int mid = (from + to) >> 1;
        constant_intervals_rec(from, mid, f, g, start, previous, f(
            mid));
        constant_intervals_rec(mid + 1, to, f, g, start, previous,
            current);
    }
}

template<class F, class G>
void constant_intervals(int from, int to, F f, G g) {
    if (to < from) return;
    int start = from;
    auto previous = f(from), current = f(to);
    constant_intervals_rec(from, to, f, g, start, previous,
        current);
    g(start, to, current);
}
```

10.2 Misc. algorithms

TernarySearch.h

Description: find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \dots < f(i) \geq \dots \geq f(b)$. to reverse which of the sides allows non-strict inequalities, change the $<$ marked with (A) to $\leq$, and reverse the loop at (B). to minimize f , change it to $>$, also at (B).

```
Usage: int ind = tern_search(1,n,[&](int i){return a[i];});
Time:  $\mathcal{O}(\log(b-a))$ 
c6c385, 11 lines

template<class F>
int tern_search(int a, int b, F f) {
    assert(a <= b);
    while (b - a >= 5) {
        int mid = (a + b) / 2;
        if (f(mid) < f(mid+1)) a = mid; // (A)
        else b = mid+1;
    }
    for (int i = a+1; i < (b+1); ++i) if (f(a) < f(i)) a = i; // (B)
    return a;
}
```

LIS.h

Description: compute 1-indexed positions of a longest increasing subsequence.

Time: $\mathcal{O}(N \log N)$

```
template<class I> vector<int> lis(const vector<I>& values) {
    if (values.empty()) return {};
    vector<int> prev((int)(values).size(), -1);
    vector<pair<I, int>> res;
    for (int i = 0; i < ((int)(values).size()); ++i) {
        // change 0  $\rightarrow$  i for longest non-decreasing subsequence
        auto it = lower_bound(begin(res), end(res), pair<I, int>{
            values[i], 0});
        if (it == res.end()) res.emplace_back(), it = res.end()-1;
        *it = {values[i], i};
        prev[i] = it == res.begin() ? -1 : (it-1)->second;
    }
    int length = (int)res.size(), cur = res.back().second;
    vector<int> ans(length);
    while (length--) ans[length] = cur + 1, cur = prev[cur];
    return ans;
}
```

FastKnapsack.h

Description: given N non-negative integer weights w and a non-negative target t, computes the maximum $S \leq t$ such that S is the sum of some subset of the weights.

Time: $\mathcal{O}(N \max(w_i))$

```
int knapsack(vector<int> w, int t) {
    int a = 0, b = 0, x;
    while (b < (int)(w).size() && a + w[b] <= t) a += w[b++];
    if (b == (int)(w).size()) return a;
    int m = *max_element(begin(w), end(w));
    vector<int> u, v(2*m, -1);
    v[a+m-t] = b;
    for (int i = b; i < ((int)(w).size()); ++i) {
        u = v;
        for (int x = 0; x < (m); ++x) v[x+w[i]] = max(v[x+w[i]], u[x]);
        for (x = 2*m; --x > m;) for (int j = max(0, u[x]); j < (v[x]); ++j)
            v[x-w[j]] = max(v[x-w[j]], j);
    }
    for (a = t; v[a+m-t] < 0; a--) ;
    return a;
}
```

10.3 Dynamic programming

KnuthDP.h

Description: when doing DP on intervals: $a[i][j] = \min_{i < k < j} (a[i][k] + a[k][j]) + f(i, j)$, where the (minimal) optimal k increases with both i and j , one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. this is known as knuth DP. sufficient criteria for this are if $f(b, c) \leq f(a, d)$ and $f(a, c) + f(b, d) \leq f(a, d) + f(b, c)$ for all $a \leq b \leq c \leq d$. consider also: LineContainer (ch. data structures), monotone queues, ternary search.

Time: $\mathcal{O}(N^2)$

DivideAndConquerDP.h

Description: given $a[i] = \min_{lo(i) \leq k \leq hi(i)} (f(i, k))$ where the (minimal) optimal k increases with i , computes $a[i]$ for the inclusive range $[L, R]$.

Time: $\mathcal{O}((N + (hi - lo)) \log N)$

```
struct DpState { // modify at will:
    int lo(int ind) { return 1; }
    int hi(int ind) { return ind; }
    ll f(int ind, int k) { return dp[ind][k]; }
    void store(int ind, int k, ll v) { res[ind] = pii(k, v); }

    void rec(int left, int right, int opt_left, int opt_right) {
        if (left > right) return;
        int mid = (left + right) >> 1;
        pair<ll, int> best(LLONG_MAX, opt_left);
        for (int k = max(opt_left, lo(mid)); k <= min(opt_right, hi(
            mid)); ++k)
            best = min(best, make_pair(f(mid, k), k));
        store(mid, best.second, best.first);
        rec(left, mid - 1, opt_left, best.second);
        rec(mid + 1, right, best.second, opt_right);
    }
    void solve(int left, int right) { rec(left, right, INT_MIN,
        INT_MAX); }
};
```

10.4 Debugging tricks

- signal(SIGSEGV, [](int) { _Exit(0); }); converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). _GLIBCXX_DEBUG failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).

- feenableexcept(29); kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.5 Optimization tricks

__builtin_ia32_ldmxcsr(40896); disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Bit hacks

- x & -x is the least bit in x.
- for (int x = m; x;) { --x &= m; ... } loops over all subset masks of m (except m itself).
- c = x&-x, r = x+c; (((r^x) >> 2)/c) | r is the next number after x with the same number of bits set.
- rep(b,0,K) rep(i,0,(1 << K)) if (i & 1 << b) D[i] += D[i^(1 << b)]; computes all sums of subsets.

10.5.2 Pragmas

- **#pragma** GCC optimize ("Ofast") will make GCC auto-vectorize loops and optimizes floating points better.
- **#pragma** GCC target ("avx2") can double performance of vectorized code, but causes crashes on old machines.
- **#pragma** GCC optimize ("trapv") kills the program on integer overflows (but is really slow).

FastMod.h

Description: compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. returns a value congruent to $a \pmod b$ in the range $[0, 2b)$.

```
751a02, 8 lines
typedef unsigned long long ull;
struct FastMod {
    ull b, m;
    FastMod(ull b) : b(b), m(-1ULL / b) {}
    ull reduce(ull a) { // a % b + (0 or b)
        return a - (ull)((__uint128_t(m) * a) >> 64) * b;
    }
};
```

FastInput.h

Description: read an integer from stdin. usage requires your program to pipe in input from file.
Usage: ./a.out < input.txt
Time: about 5x as fast as cin/scanf.

```
20c8cc, 17 lines
inline char gc() { // like getchar()
    static char buf[1 << 16];
    static size_t bc, be;
    if (bc >= be) {
        buf[0] = 0, bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
    return buf[bc++]; // returns 0 on EOF
}

int read_int() {
    int a, c;
    while ((a = gc()) < 40);
    if (a == '-') return -read_int();
    while ((c = gc()) >= 48) a = a * 10 + c - 480;
    return a - 48;
}
```

BumpAllocator.h

Description: when you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation.

```
745db2, 8 lines
// either globally or in a single class:
static char buf[450 << 20];
void* operator new(size_t s) {
    static size_t i = sizeof buf;
    assert(s < i);
    return (void*)&buf[i -= s];
}
void operator delete(void*) {}
```

SmallPtr.h

Description: A 32-bit pointer that points into bump_allocator memory.

```
2b07d6, 10 lines
template<class T> struct SmallPtr {
    unsigned ind;
```

```
SmallPtr(T* p = 0) : ind(p ? unsigned((char*)p - buf) : 0) {
    assert(ind < sizeof buf);
}
T& operator*() const { return *(T*)(buf + ind); }
T* operator->() const { return &*this; }
T& operator[](int a) const { return (&this)[a]; }
explicit operator bool() const { return ind; }
};
```

BumpAllocatorSTL.h

Description: bump_allocator for STL containers.
Usage: vector<vector<int, SmallAllocator<int>>> edges(n + 1);

```
664076, 14 lines
char buf[450 << 20] alignas(16);
size_t buf_ind = sizeof buf;

template<class T> struct SmallAllocator {
    typedef T value_type;
    SmallAllocator() {}
    template<class U> SmallAllocator(const U&) {}
    T* allocate(size_t n) {
        buf_ind -= n * sizeof(T);
        buf_ind &= 0 - alignof(T);
        return (T*)(buf + buf_ind);
    }
    void deallocate(T*, size_t) {}
};
```

SIMD.h

Description: cheat sheet of SSE/AVX intrinsics, for doing arithmetic on several numbers at once. can provide a constant factor improvement of about 4, orthogonal to loop unrolling. operations follow the pattern "`mm(256)?name.(si(128|256)|epi(8|16|32|64)|pd|ps)"`. not all are described here; grep for `mm_` in `/usr/lib/gcc/*/4.9/include/` for more. if AVX is unsupported, try 128-bit operations, "emmintrin.h" and `#define _SSE_` and `_MMX_` before including it. for aligned memory use `mm_malloc(size, 32)` or `int buf[N] alignas(32)`, but prefer `loadu/storeu`.

```
c43a4d, 46 lines
#pragma GCC target ("avx2") // or sse4.1
#include "immintrin.h"

typedef __m256i mi;
template<class T>
mi load_unaligned(T& value) {
    return _mm256_loadu_si256(reinterpret_cast<const mi*>(&value)
    );
}

// high-level/specific methods:
// load(u)?_si256, store(u)?_si256, setzero_si256, _mm_malloc
// blendv_(epi8|ps|pd) (z?y:x), movemask_epi8 (hibits of bytes)
// i32gather_epi32(addr, x, 4): map addr[] over 32-b parts of x
// sad_epu8: sum of absolute differences of u8, outputs 4xi64
// maddubs_epi16: dot product of unsigned i7's, outputs 16xi15
// madd_epi16: dot product of signed i16's, outputs 8xi32
// extractf128_si256(. i) (256->128), cvtsi128_si32 (128->lo32)
// permute2f128_si256(x,x,1) swaps 128-bit lanes
// shuffle_epi32(x, 3*64+2*16+1*4+0) == x for each lane
// shuffle_epi8(x, y) takes a vector instead of an mm

// methods that work with most data types (append e.g. _epi32):
// set1, blend (i8?x:y), add, adds (sat.), mullo, sub, and/or,
// andnot, abs, min, max, sign(1,x), cmpgt|eq, unpack(lo|hi)

int sumi32(mi m) { union {int v[8]; mi m;} u; u.m = m;
    int ret = 0; for (int i = 0; i < (8); ++i) ret += u.v[i];
    return ret; }
```

```
mi zero() { return _mm256_setzero_si256(); }
mi one() { return _mm256_set1_epi32(-1); }
bool all_zero(mi m) { return _mm256_testz_si256(m, m); }
bool all_one(mi m) { return _mm256_testc_si256(m, one()); }

ll example_filtered_dot_product(int n, short* a, short* b) {
    int i = 0; ll r = 0;
    mi zero = _mm256_setzero_si256(), acc = zero;
    while (i + 16 <= n) {
        mi va = load_unaligned(a[i]), vb = load_unaligned(b[i]); i
            += 16;
        va = _mm256_and_si256(_mm256_cmpgt_epi16(vb, va), va);
        mi vp = _mm256_madd_epi16(va, vb);
        acc = _mm256_add_epi64(_mm256_unpacklo_epi32(vp, zero),
            _mm256_add_epi64(acc, _mm256_unpackhi_epi32(vp, zero)));
    }
    union {ll v[4]; mi m;} u; u.m = acc; for (int i = 0; i < (4);
        ++i) r += u.v[i];
    for (;i<n;++i) if (a[i] < b[i]) r += a[i]*b[i]; // <- equiv
    return r;
}
```

Techniques (A)

techniques.txt	159 lines
Recursion	
Divide and conquer	
Finding interesting points in N log N	
Algorithm analysis	
Master theorem	
Amortized time complexity	
Greedy algorithm	
Scheduling	
Max contiguous subvector sum	
Invariants	
Huffman encoding	
Graph theory	
Dynamic graphs (extra book-keeping)	
Breadth first search	
Depth first search	
* Normal trees / DFS trees	
Dijkstra's algorithm	
MST: Prim's algorithm	
Bellman-Ford	
Konig's theorem and vertex cover	
Min-cost max flow	
Lovasz toggle	
Matrix tree theorem	
Maximal matching, general graphs	
Hopcroft-Karp	
Hall's marriage theorem	
Graphical sequences	
Floyd-Warshall	
Euler cycles	
Flow networks	
* Augmenting paths	
* Edmonds-Karp	
Bipartite matching	
Min. path cover	
Topological sorting	
Strongly connected components	
2-SAT	
Cut vertices, cut-edges and biconnected components	
Edge coloring	
* Trees	
Vertex coloring	
* Bipartite graphs (=> trees)	
* 3^n (special case of set cover)	
Diameter and centroid	
K'th shortest path	
Shortest cycle	
Dynamic programming	
Knapsack	
Coin change	
Longest common subsequence	
Longest increasing subsequence	
Number of paths in a dag	
Shortest path in a dag	
Dynprog over intervals	
Dynprog over subsets	
Dynprog over probabilities	
Dynprog over trees	
3^n set cover	
Divide and conquer	
Knuth optimization	
Convex hull optimizations	
RMQ (sparse table a.k.a 2^k-jumps)	
Bitonic cycle	
Log partitioning (loop over most restricted)	
Combinatorics	

Computation of binomial coefficients
Pigeon-hole principle
Inclusion/exclusion
Catalan number
Pick's theorem
Number theory
Integer parts
Divisibility
Euclidean algorithm
Modular arithmetic
* Modular multiplication
* Modular inverses
* Modular exponentiation by squaring
Chinese remainder theorem
Fermat's little theorem
Euler's theorem
Phi function
Frobenius number
Quadratic reciprocity
Pollard-Rho
Miller-Rabin
Hensel lifting
Vieta root jumping
Game theory
Combinatorial games
Game trees
Mini-max
Nim
Games on graphs
Games on graphs with loops
Grundy numbers
Bipartite games without repetition
General games without repetition
Alpha-beta pruning
Probability theory
Optimization
Binary search
Ternary search
Unimodality and convex functions
Binary search on derivative
Numerical methods
Numeric integration
Newton's method
Root-finding with binary/ternary search
Golden section search
Matrices
Gaussian elimination
Exponentiation by squaring
Sorting
Radix sort
Geometry
Coordinates and vectors
* Cross product
* Scalar product
Convex hull
Polygon cut
Closest pair
Coordinate-compression
Quadtrees
KD-trees
All segment-segment intersection
Sweeping
Discretization (convert to events and sweep)
Angle sweeping
Line sweeping
Discrete second derivatives
Strings
Longest common substring
Palindrome subsequences

Knuth-Morris-Pratt
Tries
Rolling polynomial hashes
Suffix array
Suffix tree
Aho-Corasick
Manacher's algorithm
Letter position lists
Combinatorial search
Meet in the middle
Brute-force with pruning
Best-first (A*)
Bidirectional search
Iterative deepening DFS / A*
Data structures
LCA (2^k-jumps in trees in general)
Pull/push-technique on trees
Heavy-light decomposition
Centroid decomposition
Lazy propagation
Self-balancing trees
Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
Monotone queues / monotone stacks / sliding queues
Sliding queue using 2 stacks
Persistent segment tree